

Lecture Notes

STAT 33B with Gaston Sanchez
Fall 2023

Contents

1	Wednesday, August 23rd	6
1.1	Class Syllabus	6
1.1.1	Lectures	6
2	Wednesday, August 30th	7
2.1	Vectors	7
2.2	Types	7
2.2.1	Finding Types	7
2.3	Special Values	7
2.3.1	NULL	7
2.3.2	Missing Values	7
2.3.3	Not A Number	8
2.3.4	Inf	8
2.4	Mixing Data Types	8
2.4.1	Implicit Coercion	8
2.4.2	Explicit Coercion	8
2.5	Vectorization	8
2.6	Recycling	9
3	Wednesday, September 6th	10
3.1	Vectors	10
3.1.1	Numeric Subsetting	10

3.1.2	Reassignment	10
3.1.3	Out of Bounds Indexing	10
3.1.4	Logical Subsetting	10
3.1.5	Double Brackets	10
3.2	Matrices	11
3.2.1	Matrix Representation	11
3.2.2	Matrix Storage	11
3.2.3	Changing Representation	11
3.2.4	Displaying Matrices	11
3.2.5	Style	11
4	Wednesday, September 13th	12
4.1	Atomic Objects	12
4.2	Lists	12
4.2.1	Element Names	12
4.2.2	Subsetting (Subindex, Subscript)	12
4.2.3	List Operations	12
4.3	Data Frames	12
4.3.1	Properties/Features of Data Frames	13
5	Wednesday, September 20th	14
5.1	Functions	14
5.2	Imports	14
5.2.1	Unstructured Imports	14
5.2.2	Use of these functions	14
5.3	Reading structured imports	14
5.4	Exports	15

5.5	Linear Regression	15
6	Wednesday, September 27th	16
6.1	tidyverse	16
6.2	tibble	16
6.2.1	Recycling	16
6.2.2	Column name Autocomplete	16
6.3	dplyr	17
6.3.1	slice	17
6.3.2	select	17
6.3.3	filter	17
6.3.4	mutate	17
6.3.5	arrange	17
6.3.6	summarise	18
6.3.7	group_by	18
6.4	Pipe operators	18
7	Wednesday, October 4th	19
8	Wednesday, October 11th	113
8.1	Programming Structures	113
8.2	Compound Expressions	113
8.3	If/Else	113
8.3.1	Vectorized ifelse	114
8.3.2	Testing for missing values	114
8.4	case_when	114
8.5	switch	115

8.6	Lookup vector trick	115
9	Wednesday, October 18th	116
9.1	Control Structures in R: Loops and Iterations	116
9.2	For loops	116
9.2.1	Usage with vectors and lists	116
9.2.2	Using the seq() function	116
9.3	Control statements within loops	117
9.3.1	next	117
9.3.2	break	117
9.4	Apply Family Functions	117
9.4.1	apply()	118
9.5	Anonymous Functions	118
9.5.1	Usage in apply()	118
9.6	Range Computation Using apply	118
9.7	summarise	119
9.8	while loops in R	119
10	Wednesday, October 25th	121
10.1	Variables in R	121
10.1.1	Global and Local Variables	121
10.1.2	Lexical Scoping in R	121
10.2	Names and Values in R	122
10.2.1	Understanding Variable Bindings	122
10.2.2	Copy on Modify	122
10.2.3	Binding Counting in R	123
10.3	Memory Management in R	123

10.4 Advanced R Concepts	123
10.5 R's Motto: Everything is an Object	123
10.5.1 Objects and Functions in R	123
11 Wednesday, November 1st	125
11.1 Environments in R	125
11.1.1 Understanding Environments	125
11.1.2 Properties of Environments	125
11.2 Characteristics of Environments	126
11.3 Environments and Function Execution	126
11.3.1 Role and Importance of Environments	126
11.3.2 Function Evaluation in R	126
12 Wednesday, November 8th	128

1 Wednesday, August 23rd

1.1 Class Syllabus

1.1.1 Lectures

Introduction to Advanced Programming in R (Fall 2023)

Course Information

Instructor: Gaston Sanchez, OH TBD in Evans 309

First homework will be released on September 8. Homeworks will be released every two weeks.

Midterm: October 13

Final: December 12

Labs are worth 10% of grade, homeworks are worth 65% of grade, midterm is worth 6% of grade, and final is worth 19% of grade.

No rounding up grades when calculating. This semester is different due to the significant demand.

Step 1. Install R.

Step 2. Install R Studio. (Google “Install R Studio”)

If you run into installation issues, use r.datahub.berkeley.edu.

R Studio Components

History contains all former commands in case something happens to your code. R automatically creates a `.Rhistory` file which contains these commands.

Environment keeps track of all the objects created in the session. R stores this information in an `.RData` file. This is why the console will say “Workspace loaded from `.RData`”.

2 Wednesday, August 30th

2.1 Vectors

We can make a vector as such: `v <- c(1, 2, 3)` where `c` is either the ‘combine’ or ‘concatenate’ function.

2.2 Types

R has 4 main data types. In order of size that they occupied from least to greatest:

1. **logical**
 - ex. `z = TRUE`
2. **integer**
 - ex. `y = 105L`, where the letter ‘L’ declares this as an integer
3. **dbl** (double, or ‘real’ numbers)
 - ex. `x = 105.10`
- 3.5. **complex**
4. **chr** (character)

2.2.1 Finding Types

We can quantify the size of a variable *var* with `object.size(var)` as something like 56 bytes. Furthermore we can use `typeof` and `mode`, where the latter is sometimes preferred if we do not need the extra granularity, and the former is preferred when you need to be precise.

`typeof(v[i])` where `v[i]` is NA, will give you the type of the other values in the vector `v`.

`typeof(NA)` gives `\logical"`.

2.3 Special Values

2.3.1 NULL

NULL is represented in all caps and is not a vector. Furthermore, it is a reserved keyword – you will not be able to assign to it (i.e. using it as a variable name).

2.3.2 Missing Values

These are represented by NA which stands for ‘Not Available’.

2.3.3 Not A Number

You can get NaN's from negative inputs to the `sqrt` and `log` functions.

2.3.4 Inf

You can get Inf or -Inf with `1/0` or `-1/0` respectively.

2.4 Mixing Data Types

2.4.1 Implicit Coercion

If you combine different types then you combine types with the resulting vector having the type being the most dominant value.

If you add a NULL with any other types, the NULL's will just be dropped.

2.4.2 Explicit Coercion

`as.logical(0)` returns FALSE.

`as.logical` returns TRUE for any other numeric value.

`as.logical` returns NA for any value that takes more memory than a *numeric* value type.

Likewise there are: `as.integer(...)`
`as.double(...)`
`as.character(...)`

Similarly we have testing functions `is.logical(...)`
`is.integer(...)`
`is.double(...)`
`is.character(...)`

Coercion Sanity Check:

Q: What type does everything get converted to in `c(TRUE, 1L, 4L, 6)`?

A: double

2.5 Vectorization

Doing elementwise operations all at once!

2.6 Recycling

```
> a = c(1, 2)
> b = c(2, 4, 6, 8)
> a + b
[1] 3 6 7 10
```

Note that R ‘recycles’ or ‘repeats’ the elements in the shorter vector to match the longer one.

If the shorter one’s length is a multiple of the longer one’s length then this occurs without issue.

! However if that is not the case, then we run into a Warning – though it will still execute!

3 Wednesday, September 6th

3.1 Vectors

Vectors are fundamental data structures in R. They can be subsetted, reassigned, and indexed both within and outside their bounds.

3.1.1 Numeric Subsetting

Numeric vectors can be used for subsetting in R. Exclusions can be made using both the column operator and parentheses.

3.1.2 Reassignment

To modify a value at a specific position in a vector, use brackets. For example, to change the value at the x_1 position in vector x , use: `x[x1] = a`.

3.1.3 Out of Bounds Indexing

- Accessing an index outside the vector's bounds gives an `<NA>` message.
- This can also be used to append elements. For a list with four elements, `x[5] = a` adds a as the fifth element.

3.1.4 Logical Subsetting

- Exclude an index by writing the logical vector. For a vector X with four elements, use `X[c(TRUE, TRUE, TRUE, FALSE)]` to omit the fourth value.
- For comparison, `X > 5` in `[2, 3, 6, 5]` gives `FALSE, FALSE, TRUE, FALSE`. Updating X with these omits the 1st, 2nd, and 4th values.

3.1.5 Double Brackets

- `x[[1]]` returns the first value of vector X .
- `X[[1, 2]]` gives an error as it tries to select more than one element.
- `X[[A]]` should return only one element.
- Both `X[[a]]` and `X[a]` return element a . Single brackets are common, but double brackets, which are faster, return only one element and are used for lists.

3.2 Matrices

Matrices in R are two-dimensional vectors that can store data in rows and columns.

3.2.1 Matrix Representation

- Stored in column-major format.
- Represented as: `matrix(data = c(2, 4, 6, 8), nrow = 2, ncol = 2)`.

3.2.2 Matrix Storage

- First argument is `data =`.
- Stored as vectors but can be matrix objects with rows and columns.

3.2.3 Changing Representation

- Use `byrow = TRUE` for row-major representation.
- Display the matrix by inputting its name.

3.2.4 Displaying Matrices

To view a matrix X as the vector R stores it, use `as.vector(X)`.

3.2.5 Style

For clarity, specify both `nrow` and `ncol` when defining a matrix.

4 Wednesday, September 13th

4.1 Atomic Objects

Atomic Objects include vectors (1-dimensional), matrices (2-dimensional), and n-dimensional arrays.

4.2 Lists

A vector is a sequence of continuous cells, while a list is a collection of non-continuous cells.

A list y can be represented as `y = list(a, b, c)`, with each element being its own separate vector.

4.2.1 Element Names

You can assign names to each element using the `names()` function. For our list y , we can set `names(y) = c("x", "y", "z")`, sequentially assigning names to each element. Consider `z = list(TRUE, 3L, "hi")`.

We can also rename (not the values) of each element using the list constructor:
`w = list(a = TRUE, b = 3L, c = "hi")`.

4.2.2 Subsetting (Subindex, Subscript)

1. **Single brackets:** Using single brackets, we can see both the name and the value of the item we're accessing. For example, `y[1]` evaluates to `a [1] TRUE`.
2. **Double brackets:** Using double brackets, we only see the value of the element we're accessing. For instance, `y[[1]]` evaluates to `[1] TRUE`.

4.2.3 List Operations

List operations in R are fairly limited. For instance, we cannot duplicate lists simply by multiplying with integers. Assume we have a list `a = list("this", "is", "a", "list")`. We cannot use `2 * a` to duplicate the list. Nested elements of a list can be accessed using appropriate indexing.

4.3 Data Frames

A data frame is a non-atomic version of a matrix. Data frames are initialized with:

```
dframe = data.frame(
```

```
"name" = c('Luke', 'Leia', 'Han'),  
"height" = c(177, 160, 190),  
"force" = c(TRUE, TRUE, FALSE)  
)
```

4.3.1 Properties/Features of Data Frames

When using `class()` on data frames, R will indicate that it's a data frame. However, `typeof()` on a data frame will reveal that it is essentially a list. Individual columns within a data frame can be accessed using the `$` notation, e.g., `dframe$height`. Specific elements can also be accessed using 2D indexing. For instance:

- `dframe[1, 1]` gives the element in the first row and first column.
- `dframe[1, 1:3]` provides elements in the first row across the first to third columns.
- `dframe[1:3, 1]` yields the elements in the first to third rows of the first column.

5 Wednesday, September 20th

5.1 Functions

The function `file()` allows you to open files, `url()` allows you to open URLs, and `unz()` allows you to unzip files. Most of the time, you don't need to explicitly call these functions. They are typically executed behind the scenes during the import/export process.

5.2 Imports

Tabular formats include: .csv, .tsv, and .txt. To read these data table formats, we use `read.table()`. Specific subfunctions of `read.table()` include `read.csv()`, `read.csv2`, `read.delim`, and `read.fwf()`. The output of these functions is a `data.frame`.

There are also provisions to import data from “foreign” packages such as SAS, SPSS, Stata, Matlab, etc.

5.2.1 Unstructured Imports

For unstructured data, `readLines()` reads a character vector. The output is a character vector, not a data frame.

For clarity: Functions like `read.table()` and `readLines()` are high-level, while functions they depend on, such as `scan()`, are low-level.

5.2.2 Use of these functions

The `scan()` function can be used as `scan(file = \filename.txt", what = double())`. You can assign the output to objects. The `sep = \,"` argument can be added to distinguish between elements separated by a specific character. For instance, `scan(file = \filename.filetype", what = double(), sep = \,"`) would read each value separated by a comma.

5.3 Reading structured imports

To read a table, you can use `read.table(file = filename.filetype)`. The `header` argument can be added to specify if the data has a header. For example, to assign a table to an object named `tbl` without a header and with elements separated by commas, you would use `tbl = read.table(file = filename.filetype, header = FALSE, sep = ‘,’)`.

5.4 Exports

The functions `save()` and `sink()` can be used for exporting data. The `save()` function saves your content into a binary file, which can only be opened with R. With the `file` argument, you can specify the filename. To choose a specific location in the directory, you can prepend the file path to the name, e.g., `save(tbl, file = 'Desktop/filename.filetype')`.

The `sink()` function diverts output. Instead of displaying the output in the console, you can divert it to an external file. You can call `sink()` as: `sink(file = 'output.txt')`. Any console output will then be written to `output.txt`. To stop the diversion, call `sink()` with no arguments.

5.5 Linear Regression

You can run a linear regression model on structured data using `lm()`. In the expression `reg = lm(mpg ~ cyl + disp, data = mtcars)`, `mpg` is the response variable, and `cyl` and `disp` to the right of the `~` are the explanatory variables. The `data = mtcars` argument specifies that the data comes from the built-in R `mtcars` dataset. The result is stored in the `reg` object.

6 Wednesday, September 27th

6.1 tidyverse

Tidyverse can be loaded as such:

```
1 library(tidyverse)
```

Tidyverse is a collection of packages for data science (whatever that means). There are 8 core packages included:

1. dplyr
2. ggplot2
3. readr
4. forcats
5. stringr
6. tibble
7. tidyr
8. purrr

These “tidy” packages *build* on top of base R objects but with its own base object being the **tibble**.

6.2 tibble

The **tibble** is the modern reimagining of the **data.frame**, you can think of them as mostly the same – though philosophically the **tibble** tries to be more robust and consistent.

6.2.1 Recycling

R dataframes automatically recycle all the time. **tibble** does not recycle at all and will error instead, except in the case where you say **tib\$myCol = 2** which will add a column of all 2’s.

6.2.2 Column name Autocomplete

R dataframes automatically figure out what you mean when you say **dat\$gen** to refer to a column called “gender”. **tibble** does not do this and will error unless you use the full name: **tib\$gender**.

6.3 dplyr

6.3.1 slice

Select a given row:

1. In base R we say statements like: `dat[1, ]` or `dat[1:5, ]` or etc or etc. But in `tibble` we use *functions* instead.
The `tibble` equivalents are: `slice(dat, 1)` or `slice(dat, 1:5)` or `slice(dat, c(2, 4))` or `slice(dat, -1)` to exclude the first row.

6.3.2 select

Select a given column:

1. In base R we say statements like: `dat[, 1]` or `dat[, 1:5]` or etc or etc. But in `tibble` we use *functions* instead.
The `tibble` equivalents are: `select(dat, 1)` or `select(dat, 1:5)` or `select(dat, c(2, 4))` or `select(dat, -1)` to exclude the first row.

6.3.3 filter

This allows you to filter the content of your table based on some condition:

1. In base R we say statements like: `dat[gender == 'female',]`. But in `tibble` we use *functions* instead.
The `tibble` equivalents are: `filter(dat, gender == 'female')`.

6.3.4 mutate

This `tibble` command does not modify your dataframe in-place – it adds a new column instead:

1. In base R we say statements like: `dat$height = dat$height * 100` to transform an existing column. But in `tibble` we use *functions* instead.
The `tibble` equivalents are: `mutate(dat, height = height * 100)`.

Note that `mutate(dat, height * 100)` would add a new column called `height * 100` so specifying the name is important.

6.3.5 arrange

This is one of the nicest R functions in this library, in my (the Professor's) opinion:

1. In base R we say statements like: `dat[order(dat$name), ]` to sort a column, by default it sorts by alphabetical increasing order, though we can also do the reverse. But in `tibble` we use *functions* instead.

The `tibble` equivalents are: `arrange(dat, name)`, by default it sorts by alphabetical increasing order, though `arrange(dat, desc(name))` does the reverse. You can even decide how to break ties by giving multiple columns: `arrange(dat, gender, height)`.

6.3.6 summarise

Note that the british spelling `summarize` is actually valid as well.

1. In base R we say statements like: `mean(dat$height)`.

The `tibble` equivalents are: `summarise(dat, mean(height))` or `summarise(dat, avg=mean(height))` for a nicer name.

6.3.7 group_by

`group_by(dat, col)` by itself does not do much, it just adds a line saying it is keeping track of a group. But things get interesting when we combine `group_by` with `summarize`

6.4 Pipe operators

There are 2 pipe operators: `%>%` and `|>` which means:

$$f(x, y, z) \iff x |> f(y, z) \iff x \%>\% f(y, z)$$

This can be used as such:

```
1 dat |> filter(gender == "female") |> summarise(avg = mean(height))
```

though often it will be styled as such:

```
1 dat |>
2   filter(gender == "female") |>
3   summarise(avg = mean(height))
```

7 Wednesday, October 4th

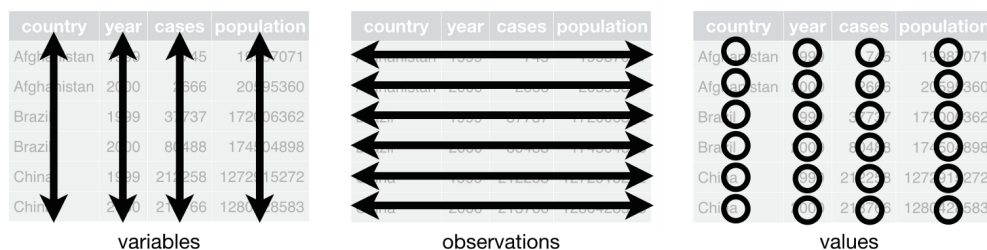
Tidyverse

- **Ordocosme** in 🇫🇷 with *Tidy* for "bien rangé" and *verse* for "univers"
- A collection of R 📦 developed by H. Wickham and others at Rstudio



Tidyverse

- "A framework for managing data that aims at making the cleaning and preparing steps [muuuuuuuch] easier" (Julien Barnier).
- Main characteristics of a tidy dataset:
 - each variable is a column
 - each observation is a row
 - each value is in a different cell



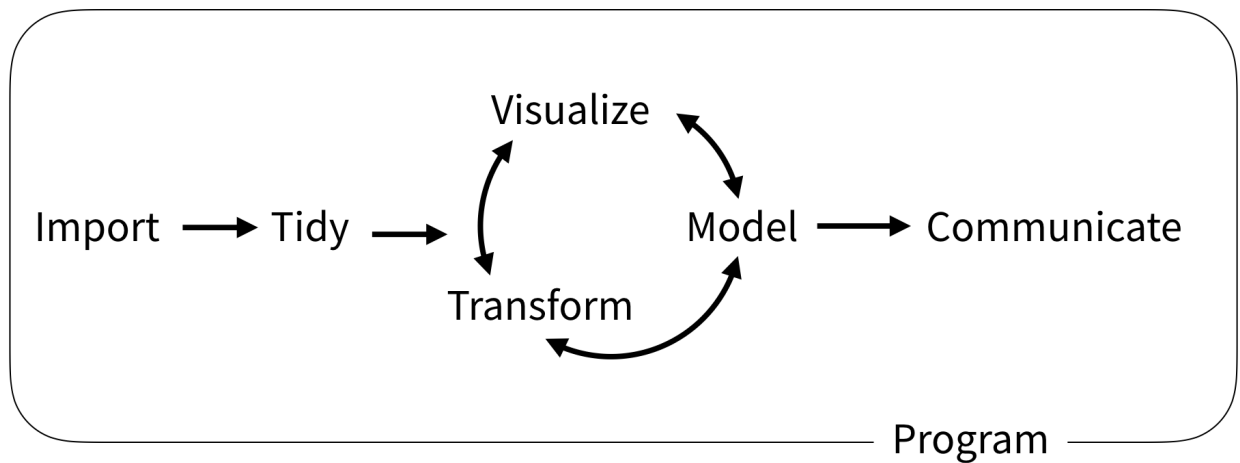
Tidyverse is a collection of R

- `ggplot2` - visualising stuff
- `dplyr`, `tidyr` - data manipulation
- `purrr` - advanced programming
- `readr` - import data
- `tibble` - improved `data.frame` format
- `forcats` - working w/ factors
- `stringr` - working w/ chain of characters

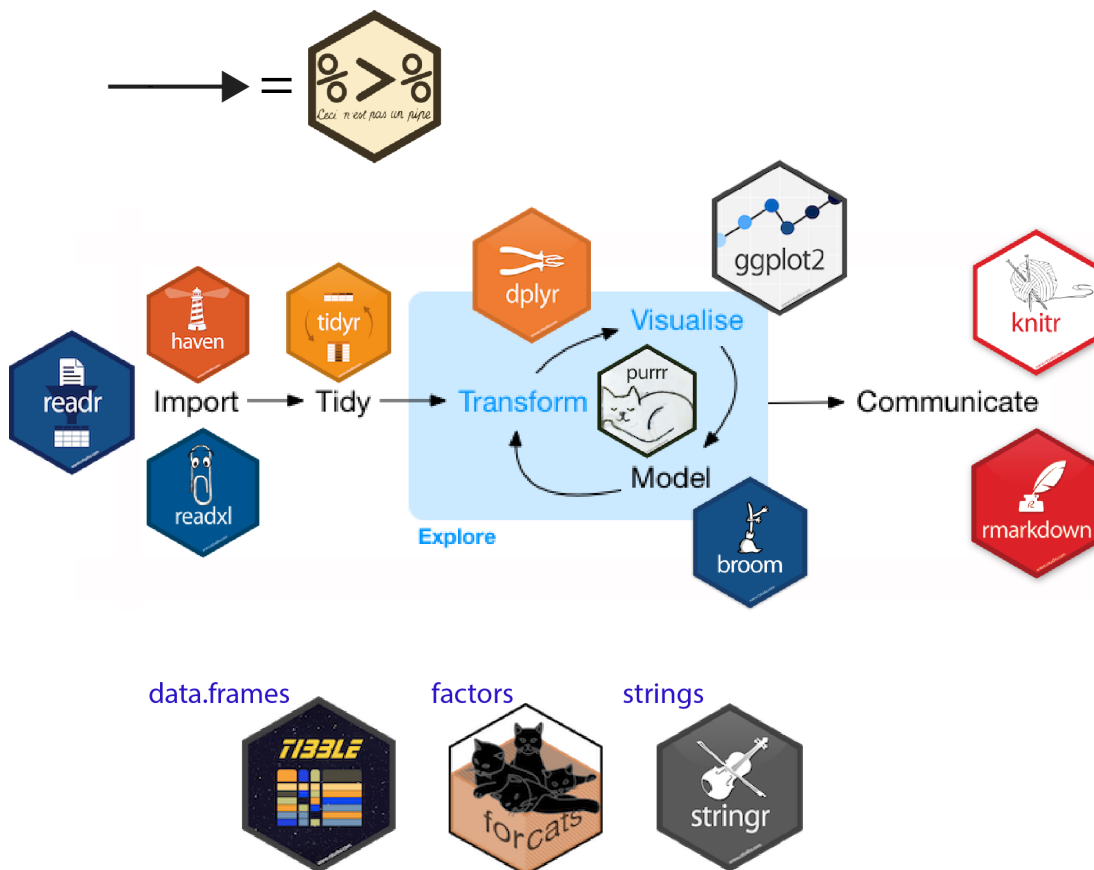
Tidyverse is a collection of R

- `ggplot2` - visualising stuff
- `dplyr`, `tidyr` - data manipulation
- `purrr` - advanced programming
- `readr` - import data
- `tibble` - improved `data.frame` format
- `forcats` - working w/ factors
- `stringr` - working w/ chain of characters

Workflow in data science



Workflow in data science, with Tidyverse



Load tidyverse

```
#install.packages("tidyverse")  
library(tidyverse)
```

Case study:

Using Twitter to predict citation rates of ecological research

plos.org create account sign in

PLOS ONE PUBLISH ABOUT BROWSE SEARCH advanced search

OPEN ACCESS PEER-REVIEWED
RESEARCH ARTICLE

Twitter Predicts Citation Rates of Ecological Research

Brandon K. Peoples , Stephen R. Midway , Dana Sackett , Abigail Lynch , Patrick B. Cooney 

Published: November 11, 2016 • <https://doi.org/10.1371/journal.pone.0166570>

120 Save	32 Citation
18,800 View	698 Share

Article	Authors	Metrics	Comments	Media Coverage
---------	---------	---------	----------	----------------

Download PDF

Import

Import data

readr::read_csv function:

- ~~keeps input types as is (no conversion to factor)~~ (since R 4.0.0)
- creates tibbles instead of data.frame
 - no names to rows
 - allows column names with special characters (see next slide)
 - more clever on screen display than w/ data.frames (see next slide)
 - **no partial matching on column names**
 - warning if attempt to access unexisting column
- is daaaaaamn fast 🏍️

Import data

```
citations_raw <- read_csv('https://raw.githubusercontent.com/oliviergimenez/intro-to-data-science/master/data/citations_raw.csv')
citations_raw
```

```
## # A tibble: 1,599 x 12
##   `Journal identi...` `5-year journal...` `Year published` Volume Issue Authors
##   <chr>                <dbl>                <dbl>    <dbl> <chr> <chr>
## 1 Ecology Letters      16.7                2014      17 12 Morin ...
## 2 Ecology Letters      16.7                2014      17 12 Jucker...
## 3 Ecology Letters      16.7                2014      17 12 Calcag...
## 4 Ecology Letters      16.7                2014      17 11 Segre ...
## 5 Ecology Letters      16.7                2014      17 11 Kaufma...
## 6 Ecology Letters      16.7                2014      17 10 Nasto ...
## 7 Ecology Letters      16.7                2014      17 10 Tschir...
## 8 Ecology Letters      16.7                2014      17 9  Barnec...
## 9 Ecology Letters      16.7                2014      17 9  Pinto-...
## 10 Ecology Letters     16.7                2014      17 9  Clough...
## # ... with 1,589 more rows, and 6 more variables: `Collection date` <chr>,
## #   `Publication date` <chr>, `Number of tweets` <dbl>, `Number of
## #   users` <dbl>, `Twitter reach` <dbl>, `Number of Web of Science
## #   citations` <dbl>
```

Tidy, transform

Rename columns

```
citations_temp <- rename(citations_raw,  
  journal = 'Journal identity',  
  impactfactor = '5-year journal impact factor',  
  pubyear = 'Year published',  
  colldate = 'Collection date',  
  pubdate = 'Publication date',  
  nbtweets = 'Number of tweets',  
  woscitations = 'Number of Web of Science citations')  
citations_temp
```

```
## # A tibble: 1,599 x 12  
##   journal impactfactor pubyear Volume Issue Authors colldate pubdate nbtweets  
##   <chr>          <dbl>   <dbl>   <dbl> <chr> <chr>   <chr>   <chr>   <dbl>  
## 1 Ecolog...      16.7    2014     17 12   Morin ... 2/1/2016 9/16/2...    18  
## 2 Ecolog...      16.7    2014     17 12   Jucker... 2/1/2016 10/13/...    15  
## 3 Ecolog...      16.7    2014     17 12   Calcag... 2/1/2016 10/21/...     5  
## 4 Ecolog...      16.7    2014     17 11   Segre ... 2/1/2016 8/28/2...     9  
## 5 Ecolog...      16.7    2014     17 11   Kaufma... 2/1/2016 8/28/2...     3  
## 6 Ecolog...      16.7    2014     17 10   Nasto ... 2/2/2016 7/28/2...    27  
## 7 Ecolog...      16.7    2014     17 10   Tschir... 2/2/2016 8/6/20...     6  
## 8 Ecolog...      16.7    2014     17 9     Barnec... 2/2/2016 6/17/2...    19  
## 9 Ecolog...      16.7    2014     17 9     Pinto-... 2/2/2016 6/12/2...    26  
## 10 Ecolog...     16.7    2014     17 9     Clough... 2/2/2016 7/17/2...    44  
## # ... with 1,589 more rows, and 3 more variables: `Number of users` <dbl>,  
## #   `Twitter reach` <dbl>, woscitations <dbl>
```

Create (or modify) columns

```
citations <- mutate(citations_temp, journal = as.factor(journal))
citations
```

```
## # A tibble: 1,599 x 12
##   journal impactfactor pubyear Volume Issue Authors colldate pubdate nbtweets
##   <fct>          <dbl>   <dbl>   <dbl> <chr> <chr>   <chr>   <chr>   <dbl>
## 1 Ecolog...      16.7     2014     17 12  Morin ... 2/1/2016 9/16/2...    18
## 2 Ecolog...      16.7     2014     17 12  Jucker... 2/1/2016 10/13/...    15
## 3 Ecolog...      16.7     2014     17 12  Calcag... 2/1/2016 10/21/...     5
## 4 Ecolog...      16.7     2014     17 11  Segre ... 2/1/2016 8/28/2...     9
## 5 Ecolog...      16.7     2014     17 11  Kaufma... 2/1/2016 8/28/2...     3
## 6 Ecolog...      16.7     2014     17 10  Nasto ... 2/2/2016 7/28/2...    27
## 7 Ecolog...      16.7     2014     17 10  Tschir... 2/2/2016 8/6/20...     6
## 8 Ecolog...      16.7     2014     17 9   Barnec... 2/2/2016 6/17/2...    19
## 9 Ecolog...      16.7     2014     17 9   Pinto-... 2/2/2016 6/12/2...    26
## 10 Ecolog...      16.7     2014     17 9   Clough... 2/2/2016 7/17/2...    44
## # ... with 1,589 more rows, and 3 more variables: `Number of users` <dbl>,
## #   `Twitter reach` <dbl>, woscitations <dbl>
```

Create (or modify) columns

```
levels(citations$journal)
```

```
## [1] "Animal Conservation"          "Conservation Letters"  
## [3] "Diversity and Distributions"   "Ecological Applications"  
## [5] "Ecology"                     "Ecology Letters"  
## [7] "Evolution"                   "Evolutionary Applications"  
## [9] "Fish and Fisheries"          "Functional Ecology"  
## [11] "Global Change Biology"        "Global Ecology and Biogeography"  
## [13] "Journal of Animal Ecology"    "Journal of Applied Ecology"  
## [15] "Journal of Biogeography"      "Limnology and Oceanography"  
## [17] "Mammal Review"               "Methods in Ecology and Evolution"  
## [19] "Molecular Ecology Resources"  "New Phytologist"
```

Give your code some air

Cleaner code with "pipe" operator %>%

```
citations_raw %>%
  rename(journal = 'Journal identity',
         impactfactor = '5-year journal impact factor',
         pubyear = 'Year published',
         colldate = 'Collection date',
         pubdate = 'Publication date',
         nbtweets = 'Number of tweets',
         woscitations = 'Number of Web of Science citations') %>%
  mutate(journal = as.factor(journal))
```

```
## # A tibble: 1,599 x 12
##   journal impactfactor pubyear Volume Issue Authors colldate pubdate nbtweets
##   <fct>         <dbl>   <dbl>  <dbl> <chr> <chr>   <chr>   <chr>   <dbl>
## 1 Ecolog...      16.7    2014    17  12  Morin ... 2/1/2016 9/16/2...    18
## 2 Ecolog...      16.7    2014    17  12  Jucker... 2/1/2016 10/13/...    15
## 3 Ecolog...      16.7    2014    17  12  Calcag... 2/1/2016 10/21/...     5
## 4 Ecolog...      16.7    2014    17  11  Segre ... 2/1/2016 8/28/2...     9
## 5 Ecolog...      16.7    2014    17  11  Kaufma... 2/1/2016 8/28/2...     3
## 6 Ecolog...      16.7    2014    17  10  Nasto ... 2/2/2016 7/28/2...    27
## 7 Ecolog...      16.7    2014    17  10  Tschir... 2/2/2016 8/6/20...     6
## 8 Ecolog...      16.7    2014    17   9  Barnec... 2/2/2016 6/17/2...    19
## 9 Ecolog...      16.7    2014    17   9  Pinto-... 2/2/2016 6/12/2...    26
## 10 Ecolog...      16.7    2014    17   9  Clough... 2/2/2016 7/17/2...    44
## # ... with 1,589 more rows, and 3 more variables: `Number of users` <dbl>,
## #   `Twitter reach` <dbl>, woscitations <dbl>
```

Name object

```
citations <- citations_raw %>%  
  rename(journal = 'Journal identity',  
         impactfactor = '5-year journal impact factor',  
         pubyear = 'Year published',  
         colldate = 'Collection date',  
         pubdate = 'Publication date',  
         nbtweets = 'Number of tweets',  
         woscitations = 'Number of Web of Science citations') %>%  
  mutate(journal = as.factor(journal))
```

Syntax with pipe

- Verb(Subject,Complement) replaced by Subject %>% Verb(Complement)
- No need to name unimportant intermediate variables
- Clear syntax (readability)



Base R from Lise Vaudor's blog

```
white_and_yolk <- crack(egg, add_seasoning)  
omelette_batter <- beat(white_and_yolk)  
omelette_with_chives <- cook(omelette_batter, add_chives)
```

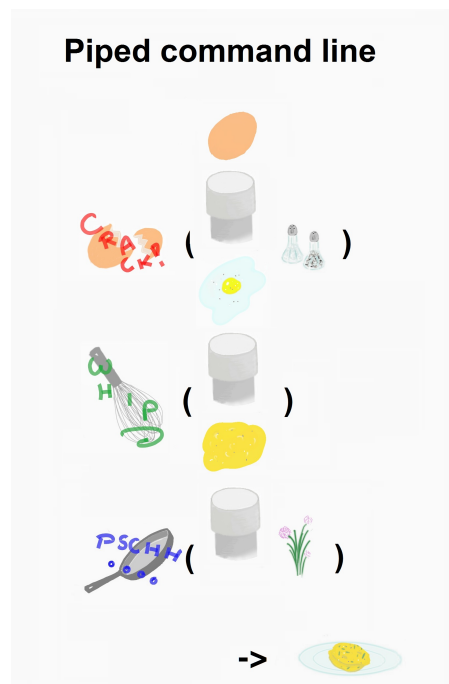
Successive command lines



@LVaudor

Piping from Lise Vaudor's blog

```
egg %>%  
  crack(add_seasoning) %>%  
  beat() %>%  
  cook(add_chives) -> omelette_with_chives
```



Tidy, transform

Select columns

```
citations %>%  
  select(journal, impactfactor, nbtweets)
```

```
## # A tibble: 1,599 x 3  
##   journal          impactfactor nbtweets  
##   <fct>          <dbl>      <dbl>  
## 1 Ecology Letters      16.7        18  
## 2 Ecology Letters      16.7        15  
## 3 Ecology Letters      16.7         5  
## 4 Ecology Letters      16.7         9  
## 5 Ecology Letters      16.7         3  
## 6 Ecology Letters      16.7        27  
## 7 Ecology Letters      16.7         6  
## 8 Ecology Letters      16.7        19  
## 9 Ecology Letters      16.7        26  
## 10 Ecology Letters     16.7        44  
## # ... with 1,589 more rows
```

Drop columns

```
citations %>%  
  select(-Volume, -Issue, -Authors)
```

```
## # A tibble: 1,599 x 9  
##   journal impactfactor pubyear colldate pubdate nbtweets `Number of user...  
##   <fct>          <dbl>   <dbl> <chr>      <chr>      <dbl>          <dbl>  
## 1 Ecolog...      16.7     2014 2/1/2016 9/16/2...      18           16  
## 2 Ecolog...      16.7     2014 2/1/2016 10/13/...     15           12  
## 3 Ecolog...      16.7     2014 2/1/2016 10/21/...      5            4  
## 4 Ecolog...      16.7     2014 2/1/2016 8/28/2...      9            8  
## 5 Ecolog...      16.7     2014 2/1/2016 8/28/2...      3            3  
## 6 Ecolog...      16.7     2014 2/2/2016 7/28/2...     27           23  
## 7 Ecolog...      16.7     2014 2/2/2016 8/6/20...      6            6  
## 8 Ecolog...      16.7     2014 2/2/2016 6/17/2...     19           18  
## 9 Ecolog...      16.7     2014 2/2/2016 6/12/2...     26           23  
## 10 Ecolog...     16.7     2014 2/2/2016 7/17/2...     44           42  
## # ... with 1,589 more rows, and 2 more variables: `Twitter reach` <dbl>,  
## #   woscitations <dbl>
```

Split a column in several columns

```
citations %>%  
  separate(pubdate, c('month', 'day', 'year'), '/')
```

```
## # A tibble: 1,599 x 14  
##   journal impactfactor pubyear Volume Issue Authors colldate month day   year  
##   <fct>          <dbl>   <dbl>  <dbl> <chr>  <chr>   <chr>   <chr> <chr> <chr>  
## 1 Ecolog...      16.7     2014    17 12  Morin ... 2/1/2016 9    16   2014  
## 2 Ecolog...      16.7     2014    17 12  Jucker... 2/1/2016 10   13   2014  
## 3 Ecolog...      16.7     2014    17 12  Calcag... 2/1/2016 10   21   2014  
## 4 Ecolog...      16.7     2014    17 11  Segre ... 2/1/2016 8     28   2014  
## 5 Ecolog...      16.7     2014    17 11  Kaufma... 2/1/2016 8     28   2014  
## 6 Ecolog...      16.7     2014    17 10  Nasto ... 2/2/2016 7     28   2014  
## 7 Ecolog...      16.7     2014    17 10  Tschir... 2/2/2016 8      6   2014  
## 8 Ecolog...      16.7     2014    17 9    Barnec... 2/2/2016 6     17   2014  
## 9 Ecolog...      16.7     2014    17 9    Pinto-... 2/2/2016 6     12   2014  
## 10 Ecolog...     16.7     2014    17 9    Clough... 2/2/2016 7     17   2014  
## # ... with 1,589 more rows, and 4 more variables: nbtweets <dbl>, `Number of  
## #   users` <dbl>, `Twitter reach` <dbl>, woscitations <dbl>
```

Transform in Date format...

```
library(lubridate)
citations %>%
  mutate(pubdate = mdy(pubdate),
         colldate = mdy(colldate))
```

```
## # A tibble: 1,599 x 12
##   journal impactfactor pubyear Volume Issue Authors colldate pubdate
##   <fct>          <dbl>   <dbl>   <dbl> <chr> <chr>   <date>   <date>
## 1 Ecolog...      16.7     2014     17 12 Morin ... 2016-02-01 2014-09-16
## 2 Ecolog...      16.7     2014     17 12 Jucker... 2016-02-01 2014-10-13
## 3 Ecolog...      16.7     2014     17 12 Calcag... 2016-02-01 2014-10-21
## 4 Ecolog...      16.7     2014     17 11 Segre ... 2016-02-01 2014-08-28
## 5 Ecolog...      16.7     2014     17 11 Kaufma... 2016-02-01 2014-08-28
## 6 Ecolog...      16.7     2014     17 10 Nasto ... 2016-02-02 2014-07-28
## 7 Ecolog...      16.7     2014     17 10 Tschir... 2016-02-02 2014-08-06
## 8 Ecolog...      16.7     2014     17 9   Barnec... 2016-02-02 2014-06-17
## 9 Ecolog...      16.7     2014     17 9   Pinto-... 2016-02-02 2014-06-12
## 10 Ecolog...     16.7     2014     17 9   Clough... 2016-02-02 2014-07-17
## # ... with 1,589 more rows, and 4 more variables: nbtweets <dbl>, `Number of
## #   users` <dbl>, `Twitter reach` <dbl>, woscitations <dbl>
```

...for easy manipulation of dates

```
library(lubridate)
citations %>%
  mutate(pubdate = mdy(pubdate),
         colldate = mdy(colldate),
         pubyear2 = year(pubdate))
```

```
## # A tibble: 1,599 x 13
##   journal impactfactor pubyear Volume Issue Authors colldate pubdate
##   <fct>          <dbl>   <dbl>   <dbl> <chr> <chr>   <date>   <date>
## 1 Ecolog...      16.7     2014     17 12 Morin ... 2016-02-01 2014-09-16
## 2 Ecolog...      16.7     2014     17 12 Jucker... 2016-02-01 2014-10-13
## 3 Ecolog...      16.7     2014     17 12 Calcag... 2016-02-01 2014-10-21
## 4 Ecolog...      16.7     2014     17 11 Segre ... 2016-02-01 2014-08-28
## 5 Ecolog...      16.7     2014     17 11 Kaufma... 2016-02-01 2014-08-28
## 6 Ecolog...      16.7     2014     17 10 Nasto ... 2016-02-02 2014-07-28
## 7 Ecolog...      16.7     2014     17 10 Tschir... 2016-02-02 2014-08-06
## 8 Ecolog...      16.7     2014     17 9 Barnec... 2016-02-02 2014-06-17
## 9 Ecolog...      16.7     2014     17 9 Pinto-... 2016-02-02 2014-06-12
## 10 Ecolog...     16.7     2014     17 9 Clough... 2016-02-02 2014-07-17
## # ... with 1,589 more rows, and 5 more variables: nbtweets <dbl>, `Number of
## #   users` <dbl>, `Twitter reach` <dbl>, woscitations <dbl>, pubyear2 <dbl>
```

- Check out `?lubridate::lubridate` for more functions

How to join tables together?

More [#dplyr](#) 🛠️ gifs! It took me a hella long time to wrap my head around the different types of joins when I first started learning them, so here's a few examples with some excellent mini datasets from [#dplyr](#) designed specifically for this purpose! [#rstats](#) [#tidyverse](#) pic.twitter.com/G56fWmIZSq

— Nic Crane (@nic_crane) 18 novembre 2018

 Watch the video

<https://www.garrickadenbuie.com/project/tidyexplain/>

`left_join(x, y)`

1	x1	1	y1
2	x2	2	y2
3	x3	4	y4

Easy character manipulation

Select rows corresponding to papers with more than 3 authors

```
citations %>%  
  filter(str_detect(Authors, 'et al'))
```

```
## # A tibble: 1,280 x 12  
##   journal impactfactor pubyear Volume Issue Authors colldate pubdate nbtweets  
##   <fct>          <dbl>   <dbl>   <dbl> <chr> <chr>   <chr>   <chr>   <dbl>  
## 1 Ecolog...      16.7     2014     17 12   Morin ... 2/1/2016 9/16/2...    18  
## 2 Ecolog...      16.7     2014     17 12   Jucker... 2/1/2016 10/13/...    15  
## 3 Ecolog...      16.7     2014     17 12   Calcag... 2/1/2016 10/21/...     5  
## 4 Ecolog...      16.7     2014     17 11   Segre ... 2/1/2016 8/28/2...     9  
## 5 Ecolog...      16.7     2014     17 11   Kaufma... 2/1/2016 8/28/2...     3  
## 6 Ecolog...      16.7     2014     17 10   Nasto ... 2/2/2016 7/28/2...    27  
## 7 Ecolog...      16.7     2014     17 10   Tschir... 2/2/2016 8/6/20...     6  
## 8 Ecolog...      16.7     2014     17 9     Barnec... 2/2/2016 6/17/2...    19  
## 9 Ecolog...      16.7     2014     17 9     Pinto-... 2/2/2016 6/12/2...    26  
## 10 Ecolog...     16.7     2014     17 9     Clough... 2/2/2016 7/17/2...    44  
## # ... with 1,270 more rows, and 3 more variables: `Number of users` <dbl>,  
## #   `Twitter reach` <dbl>, woscitations <dbl>
```

Get column with rows corresponding to papers with more than 3 authors

```
citations %>%  
  filter(str_detect(Authors, 'et al')) %>%  
  select(Authors)
```

```
## # A tibble: 1,280 x 1  
##   Authors  
##   <chr>  
## 1 Morin et al  
## 2 Jucker et al  
## 3 Calcagno et al  
## 4 Segre et al  
## 5 Kaufman et al  
## 6 Nasto et al  
## 7 Tschirren et al  
## 8 Barnechi et al  
## 9 Pinto-Sanchez et al  
## 10 Clough et al  
## # ... with 1,270 more rows
```

Select rows corresponding to papers with less than 3 authors

```
citations %>%  
  filter(!str_detect(Authors, 'et al'))
```

```
## # A tibble: 319 x 12  
##   journal impactfactor pubyear Volume Issue Authors colldate pubdate nbtweets  
##   <fct>          <dbl>   <dbl>   <dbl> <chr>  <chr>   <chr>   <chr>      <dbl>  
## 1 Ecolog...      16.7     2014     17 6    Neutle... 2/15/20... 3/17/2...      8  
## 2 Ecolog...      16.7     2014     17 5    Kellne... 2/15/20... 2/20/2...     18  
## 3 Ecolog...      16.7     2014     17 4    Griffi... 2/15/20... 1/16/2...      4  
## 4 Ecolog...      16.7     2014     17 3    Gremer... 2/15/20... 1/17/2...      4  
## 5 Ecolog...      16.7     2014     17 2    Cavier... 2/15/20... 10/17/...     16  
## 6 Ecolog...      16.7     2014     17 2    Haegma... 2/15/20... 12/5/2...      9  
## 7 Ecolog...      16.7     2013     16 12   Kearney  2/15/20... 10/1/2...     13  
## 8 Ecolog...      16.7     2013     16 9    Locey ... 2/15/20... 7/15/2...     28  
## 9 Ecolog...      16.7     2013     16 8    Quinte... 2/15/20... 6/26/2...    120  
## 10 Ecolog...     16.7     2013     16 3    Lesser... 2/15/20... 12/22/...      9  
## # ... with 309 more rows, and 3 more variables: `Number of users` <dbl>, `Twitter  
## #   reach` <dbl>, woscitations <dbl>
```

Get column with rows corresponding to papers with less than 3 authors

```
citations %>%  
  filter(!str_detect(Authors, 'et al')) %>%  
  select(Authors)
```

```
## # A tibble: 319 x 1  
##   Authors  
##   <chr>  
## 1 Neutle and Thorne  
## 2 Kellner and Asner  
## 3 Griffin and Willi  
## 4 Gremer and Venable  
## 5 Cavieres  
## 6 Haegman and Loreau  
## 7 Kearney  
## 8 Locey and White  
## 9 Quintero and Weins  
## 10 Lesser and Jackson  
## # ... with 309 more rows
```

Get column with rows corresponding to papers with less than 3 authors

```
citations %>%  
  filter(!str_detect(Authors, 'et al')) %>%  
  pull(Authors) %>%  
  head(10)
```

```
## [1] "Neutle and Thorne" "Kellner and Asner" "Griffin and Willi"  
## [4] "Gremer and Venable" "Cavieres" "Haegman and Loreau"  
## [7] "Kearney" "Locey and White" "Quintero and Weins"  
## [10] "Lesser and Jackson"
```

Select rows corresponding to papers with less than 3 authors in journal with IF < 5

```
citations %>%  
  filter(!str_detect(Authors, 'et al'), impactfactor < 5)
```

```
## # A tibble: 77 x 12  
##   journal impactfactor pubyear Volume Issue Authors colldate pubdate nbtweets  
##   <fct>          <dbl>   <dbl>   <dbl> <chr> <chr>   <chr>   <chr>   <dbl>  
## 1 Molecu...      4.9     2014     14 6     Gautier 2/27/20... 5/14/2...     2  
## 2 Molecu...      4.9     2014     14 5     Gambel... 2/27/20... 3/7/20...     7  
## 3 Molecu...      4.9     2014     14 4     Kekkono... 2/27/20... 3/10/2...     4  
## 4 Molecu...      4.9     2014     14 3     Bhatta... 2/27/20... 12/8/2...     0  
## 5 Molecu...      4.9     2014     14 1     Christ... 2/28/20... 10/25/...     0  
## 6 Molecu...      4.9     2013     13 4     Villar... 2/28/20... 5/2/20...     0  
## 7 Molecu...      4.9     2013     13 4     Wang     2/28/20... 4/25/2...     0  
## 8 Molecu...      4.9     2012     12 1     Joly     2/28/20... 9/7/20...     3  
## 9 Animal...      3.21    2014     17 6     Plavsic  2/9/2016  4/17/2...     9  
## 10 Animal...      3.21    2014     17 Supp... Knox a... 2/11/20... 11/13/...     1  
## # ... with 67 more rows, and 3 more variables: `Number of users` <dbl>, `Twitter  
## #   reach` <dbl>, woscitations <dbl>
```

Convert words to lowercase

```
citations %>%  
  mutate(authors_lowercase = str_to_lower(Authors)) %>%  
  select(authors_lowercase)
```

```
## # A tibble: 1,599 x 1  
##   authors_lowercase  
##   <chr>  
## 1 morin et al  
## 2 jucker et al  
## 3 calcagno et al  
## 4 segre et al  
## 5 kaufman et al  
## 6 nasto et al  
## 7 tschirren et al  
## 8 barnechi et al  
## 9 pinto-sanchez et al  
## 10 clough et al  
## # ... with 1,589 more rows
```

Remove all spaces in journal names

```
citations %>%  
  mutate(journal = str_remove_all(journal, " ")) %>%  
  select(journal) %>%  
  unique() %>%  
  head(5)
```

```
## # A tibble: 5 x 1  
##   journal  
##   <chr>  
## 1 EcologyLetters  
## 2 GlobalChangeBiology  
## 3 GlobalEcologyandBiogeography  
## 4 MolecularEcologyResources  
## 5 DiversityandDistributions
```

Explore stringr and regular expressions

- Check out the [vignette on stringr](#) for more examples on character manipulation and pattern matching functions.
- Check out the [vignette on regular expressions](#) which are a concise and flexible tool for describing patterns in strings.

Basic exploratory data analysis

Count

```
citations %>% count(journal, sort = TRUE)
```

```
## # A tibble: 20 x 2
##   journal      n
##   <fct>      <int>
## 1 New Phytologist 144
## 2 Ecology        108
## 3 Evolution       108
## 4 Global Change Biology 108
## 5 Global Ecology and Biogeography 108
## 6 Journal of Biogeography 108
## 7 Ecology Letters 106
## 8 Diversity and Distributions 105
## 9 Animal Conservation 102
## 10 Methods in Ecology and Evolution 90
## 11 Evolutionary Applications 74
## 12 Functional Ecology 54
## 13 Journal of Animal Ecology 54
## 14 Journal of Applied Ecology 54
## 15 Limnology and Oceanography 54
## 16 Molecular Ecology Resources 54
## 17 Conservation Letters 53
## 18 Ecological Applications 48
## 19 Fish and Fisheries 36
## 20 Mammal Review 31
```

Count

```
citations %>%  
  count(journal, pubyear) %>%  
  head()
```

```
## # A tibble: 6 x 3  
##   journal          pubyear      n  
##   <fct>          <dbl> <int>  
## 1 Animal Conservation    2012     18  
## 2 Animal Conservation    2013     18  
## 3 Animal Conservation    2014    66  
## 4 Conservation Letters   2012     17  
## 5 Conservation Letters   2013     18  
## 6 Conservation Letters   2014     18
```

Count sum of tweets per journal

```
citations %>%  
  count(journal, wt = nbtweets, sort = TRUE)
```

```
## # A tibble: 20 x 2  
##   journal          n  
##   <fct>          <dbl>  
## 1 Ecology Letters 1538  
## 2 Animal Conservation 1268  
## 3 Journal of Applied Ecology 1012  
## 4 Methods in Ecology and Evolution 699  
## 5 Global Change Biology 613  
## 6 Conservation Letters 542  
## 7 New Phytologist 509  
## 8 Global Ecology and Biogeography 379  
## 9 Ecology 335  
## 10 Evolution 335  
## 11 Journal of Animal Ecology 323  
## 12 Fish and Fisheries 261  
## 13 Evolutionary Applications 238  
## 14 Journal of Biogeography 209  
## 15 Diversity and Distributions 200  
## 16 Mammal Review 166  
## 17 Functional Ecology 155  
## 18 Molecular Ecology Resources 139  
## 19 Ecological Applications 125  
## 20 Limnology and Oceanography 0
```

Group by variable to calculate stats

```
citations %>%  
  group_by(journal) %>%  
  summarise(avg_tweets = mean(nbtweets)) %>%  
  head(10)
```

```
## # A tibble: 10 x 2  
##   journal          avg_tweets  
##   <fct>          <dbl>  
## 1 Animal Conservation      12.4  
## 2 Conservation Letters    10.2  
## 3 Diversity and Distributions  1.90  
## 4 Ecological Applications  2.60  
## 5 Ecology                 3.10  
## 6 Ecology Letters        14.5  
## 7 Evolution               3.10  
## 8 Evolutionary Applications  3.22  
## 9 Fish and Fisheries       7.25  
## 10 Functional Ecology       2.87
```

Order stuff

```
citations %>%  
  group_by(journal) %>%  
  summarise(avg_tweets = mean(nbtweets)) %>%  
  arrange(desc(avg_tweets)) %>% # decreasing order (wo desc for increasing)  
  head(10)
```

```
## # A tibble: 10 x 2  
##   journal          avg_tweets  
##   <fct>          <dbl>  
## 1 Journal of Applied Ecology    18.7  
## 2 Ecology Letters               14.5  
## 3 Animal Conservation           12.4  
## 4 Conservation Letters          10.2  
## 5 Methods in Ecology and Evolution  7.77  
## 6 Fish and Fisheries             7.25  
## 7 Journal of Animal Ecology       5.98  
## 8 Global Change Biology           5.68  
## 9 Mammal Review                  5.35  
## 10 New Phytologist               3.53
```

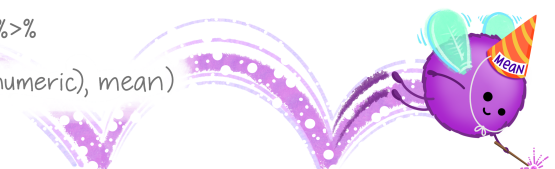
What if we want to work on several columns?

dplyr::across()

use within `mutate()` or `summarize()` to apply function(s) to a selection of columns!

EXAMPLE:

```
df %>%  
  group_by(species) %>%  
  summarize(  
    across(where(is.numeric), mean)  
  )
```



species	mass_g	age_yr	range_sqmi
pika	163	2.4	0.46
marmot	1509	3.0	0.87
marmot	2417	5.6	0.62

@allison_horst

Compute mean across all numeric columns for each journal

```
citations %>%  
  group_by(journal) %>%  
  summarize(across(where(is.numeric), mean)) %>%  
  head()
```

```
## `summarise()` ungrouping output (override with `.groups` argument)
```

```
## # A tibble: 6 x 8  
##   journal impactfactor pubyear Volume nbtweets `Number of user...` `Twitter reach`  
##   <fct>          <dbl>   <dbl> <dbl>    <dbl>          <dbl>      <dbl>  
## 1 Animal...      3.21    2013.  16.5    12.4           9.71      28345.  
## 2 Conser...      6.4     2013.   6.02   10.2           8.85      23234.  
## 3 Divers...      5.4     2013    19      1.90           1.77       2350.  
## 4 Ecolog...      5.06    2013    23      2.60           2.5       5727.  
## 5 Ecology       6.16    2013    94      3.10           2.87       6176.  
## 6 Ecolog...     16.7    2013.  16.0    14.5          14.0      44748.  
## # ... with 1 more variable: woscitations <dbl>
```

<https://github.com/courtiol/Rguides>

Data Transformation with **dplyr** 1.0 (part 2)

A guide to using `(c_)across()` to apply the same functions repeatedly

© R Data Berlin [@rdataberlin](https://github.com/courtiol/Rguides) <https://github.com/courtiol/Rguides>



Use `across()` to apply the same function(s) on multiple columns

```
tbl %>%
  [group_by(name_grouping_col_X, name_grouping_col_Y...) %>%] ## grouping is optional
  verb*(across(selected_columns, fn_with_args, [.names = prototype_for_new_col_names!])) ## prototype for naming is optional
```

selection from column:

- `mutate` (circle)
- `transmute` (right arrow)
- `group_by` (star)
- `arrange` (up arrow)
- `filter` (star)
- `summarize` (right arrow)
- `count` (up arrow)

names:

- `name_col_X:name_col_Y` (hand icon)
- `c(name_col_X, name_col_Y)` (circle)
- `all_of/any_of(c("name_col_X", "name_col_Y"))`
- `starts_with/ends_with/contains("partial_name_col")` (right arrow)
- `matches("regular_expression")` (star)
- `num_range("name_col_without_number", vector_numbers)`
- `where(fn_returning_TRUE_FALSE_for_each_col)` (star)
- `c(position_col_X, position_col_Y)`
- `everything()` (right arrow)
- `last_col()`

functions:

- `fn [, args]` (circle)
- `~ fn(.x [, args])` (star)
- `list(suffix1_new_col_names* = fn1, suffix2_new_col_names* = fn2), [args]` (star)
- `list(suffix1_new_col_names* = ~ fn1(.x [, args]), suffix2_new_col_names* = ~ fn2(.x [, args]))` (star)

EXAMPLES for across()

```
## turning specific columns into z-scores:
iris %>%
  mutate(across(c("Sepal.Length", "Petal.Width"), scale, .names = "{col}_z"))

## keeping only rows where numeric values are > 2 in all numeric columns:
iris %>% filter(across(where(is.numeric), ~ .x > 2))

## defining 3 groups for all columns with name made of 2 words around a point:
iris %>% group_by(across(matches("\\w\\w\\w"), list(discretize = cut, breaks = 3)))

## counting NA in each column:
iris %>% summarise(across(everything(), list("NA" = ~ sum(is.na(.x)))))
```

Use `c_across()` to apply the same function across multiple columns within each row

```
tbl %>%
  rowwise() %>%
  verb*(c_across(selected_columns, [args])) %>%
  ungroup()
  mutate
  transmute
  filter
  count
```

EXAMPLES for c_across()

```
## computing the area of petal (approximated as rectangles) and only keep that:
iris %>%
  rowwise() %>%
  transmute(Petal.Area = prod(c_across(contains("Petal")))) %>% ungroup()

## counting rows where at least one numeric values is > 6 for a range of columns:
iris %>%
  rowwise() %>%
  count(any(c_across(Sepal.Length:Petal.Width) > 6)) %>% ungroup()
```

Footnote: unlike `fn_with_args` in `across()` calls, the function `fn` is not here provided as a definition but used directly, so you can only use `one` function that has already been defined (you cannot define it on the fly using `~ & .x`)

Tidying tibbles

wide

id	x	y	z
1	a	c	e
2	b	d	f

long

id	key	val
1	x	a
2	x	b
1	y	c
2	y	d
1	z	e
2	z	f

Going from **long** to **wide** format and vice-versa

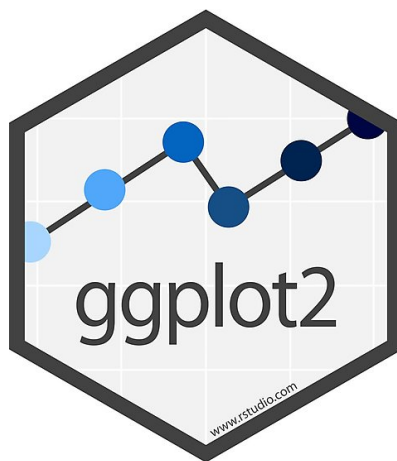
wide

	wide		
id	x	y	z
1	a	c	e
2	b	d	f

Visualize

Visualization with ggplot2

- The package ggplot2 implements a **g**rammar of **g**raphics
- Operates on data.frames or tibbles, not vectors like base R
- Explicitly differentiates between the data and its representation



The ggplot2 grammar

Grammar element

Data

Geometrics

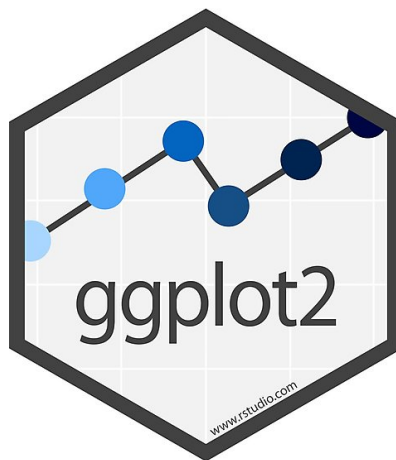
Aesthetics

What it is

The data frame being plotted

The geometric shape that will represent the data (e.g., point, boxplot, histogram)

The aesthetics of the geometric object (e.g., color, size, shape)



Scatterplots

```
citations %>%  
  ggplot() +  
  aes(x = nbtweets, y = woscitations) +  
  geom_point()
```

- Pass in the data frame as your first argument

Scatterplots

```
citations %>%  
  ggplot() +  
  aes(x = nbtweets, y = woscitations) +  
  geom_point()
```

- Pass in the data frame as your first argument
- Aesthetics maps the data onto plot characteristics, here x and y axes

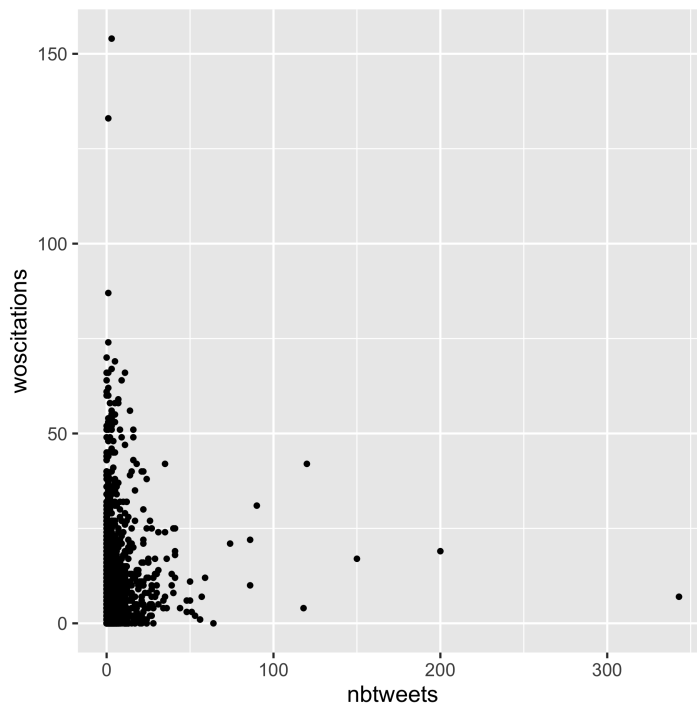
Scatterplots

```
citations %>%  
  ggplot() +  
  aes(x = nbtweets, y = woscitations) +  
  geom_point()
```

- Pass in the data frame as your first argument
- Aesthetics maps the data onto plot characteristics, here x and y axes
- Display the data geometrically as points

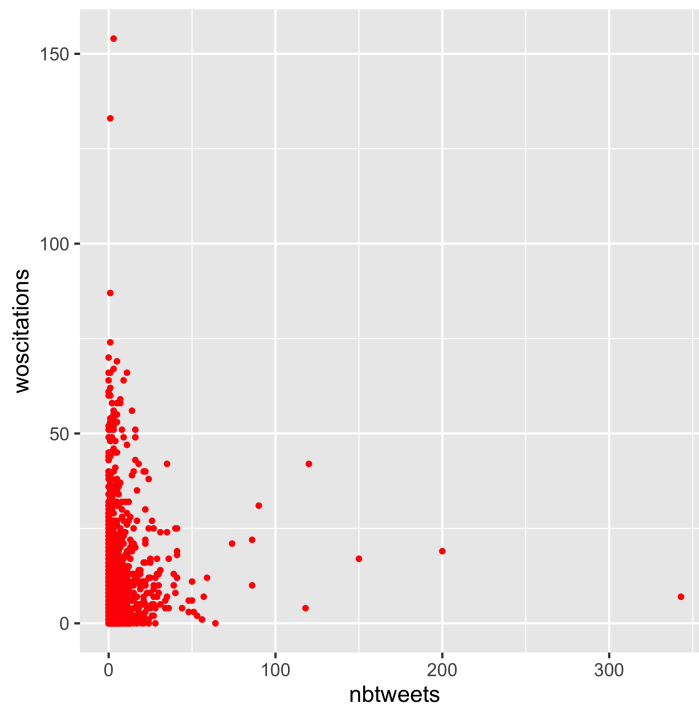
Scatterplots

```
citations %>%  
  ggplot() +  
  aes(x = nbtweets, y = woscitations) +  
  geom_point()
```



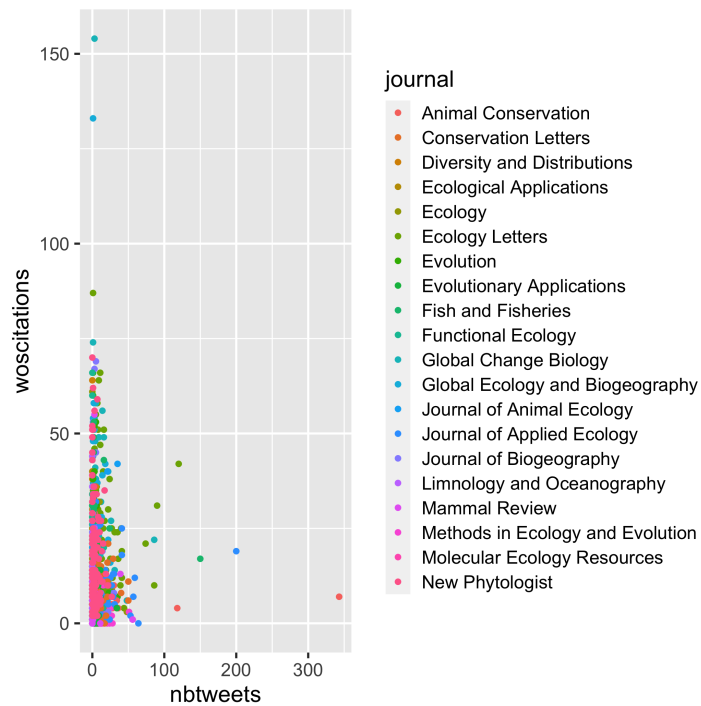
Scatterplots, with colors

```
citations %>%  
  ggplot() +  
  aes(x = nbtweets, y = woscitations) +  
  geom_point(color = "red")
```



Scatterplots, with species-specific colors

```
citations %>%  
  ggplot() +  
  aes(x = nbtweets, y = woscitations, color = journal) +  
  geom_point()
```



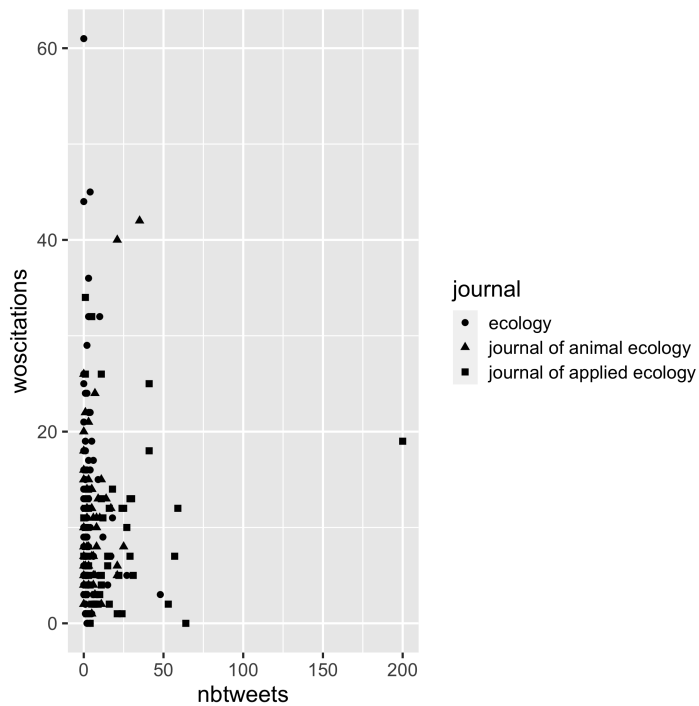
Pick a few journals

```
citations_ecology <- citations %>%  
  mutate(journal = str_to_lower(journal)) %>% # all journals names lowercase  
  filter(journal %in%  
    c('journal of animal ecology', 'journal of applied ecology', 'ecology'))  
citations_ecology
```

```
## # A tibble: 216 x 12  
##   journal impactfactor pubyear Volume Issue Authors colldate pubdate nbtweets  
##   <chr>          <dbl>   <dbl>   <dbl> <chr> <chr>   <chr>   <chr>   <dbl>  
## 1 ecology          6.16    2014     95  12   Maglia... 3/19/20... 12/1/2...     1  
## 2 ecology          6.16    2014     95  12   Soinen   3/19/20... 12/1/2...     6  
## 3 ecology          6.16    2014     95  12   Graham... 3/19/20... 12/1/2...     1  
## 4 ecology          6.16    2014     95  11   White ... 3/19/20... 11/1/2...     9  
## 5 ecology          6.16    2014     95  11   Einars... 3/19/20... 11/1/2...    15  
## 6 ecology          6.16    2014     95  11   Haav a... 3/19/20... 11/1/2...     2  
## 7 ecology          6.16    2014     95  10   Dodds ... 3/19/20... 10/1/2...     1  
## 8 ecology          6.16    2014     95  10   Brown ... 3/19/20... 10/1/2...     1  
## 9 ecology          6.16    2014     95  10   Wright... 3/19/20... 10/1/2...     0  
## 10 ecology         6.16    2014     95   9   Ramahl... 3/19/20... 9/1/20...    27  
## # ... with 206 more rows, and 3 more variables: `Number of users` <dbl>, `Twitter  
## #   reach` <dbl>, woscitations <dbl>
```

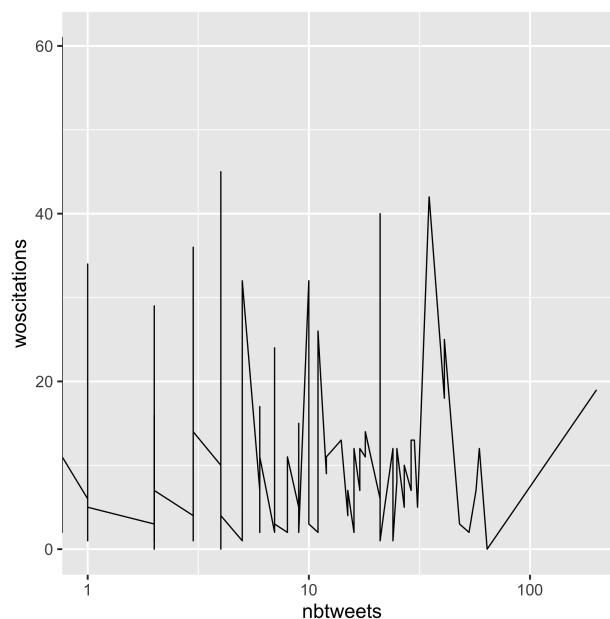
Scatterplots, with species-specific shapes

```
citations_ecology %>%  
  ggplot() +  
  aes(x = nbtweets, y = woscitations, shape = journal) +  
  geom_point(size=2)
```



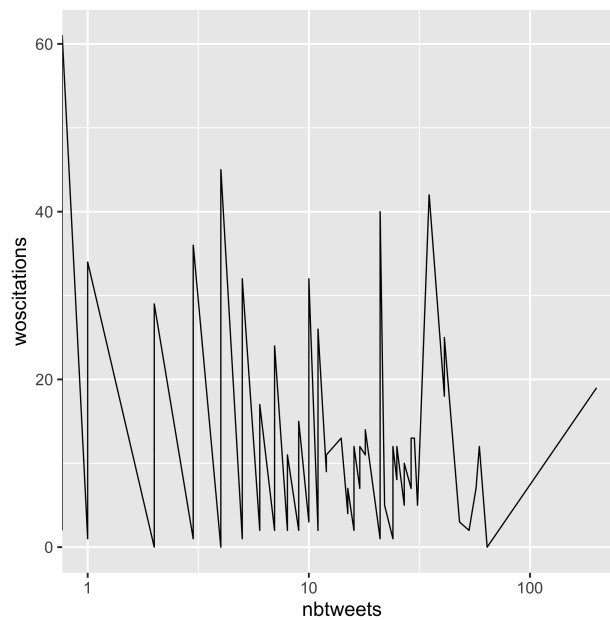
Scatterplots, lines instead of points

```
citations_ecology %>%  
  ggplot() +  
  aes(x = nbtweets, y = woscitations) +  
  geom_line() +  
  scale_x_log10()
```



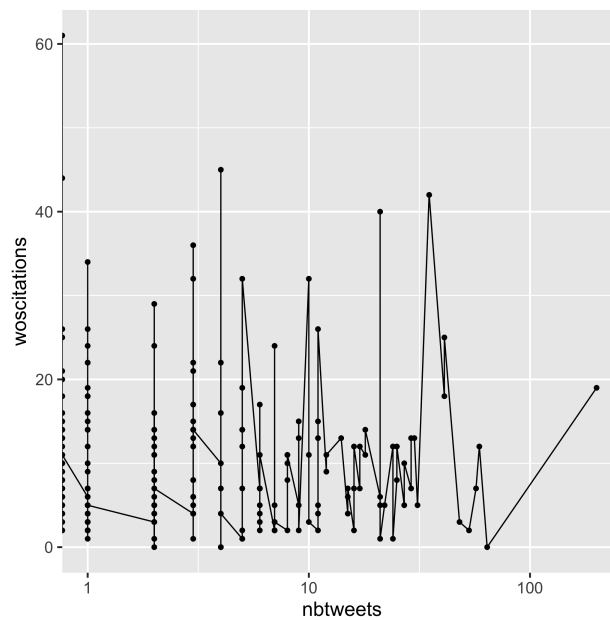
Scatterplots, lines with sorting beforehand

```
citations_ecology %>%  
  arrange(woscitations) %>%  
  ggplot() +  
  aes(x = nbtweets, y = woscitations) +  
  geom_line() +  
  scale_x_log10()
```



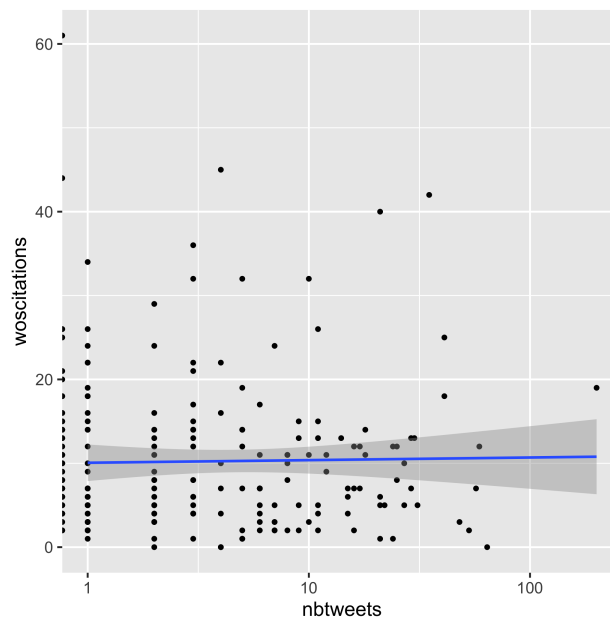
Scatterplots, add points

```
citations_ecology %>%  
  ggplot() +  
  aes(x = nbtweets, y = woscitations) +  
  geom_line() +  
  geom_point() +  
  scale_x_log10()
```



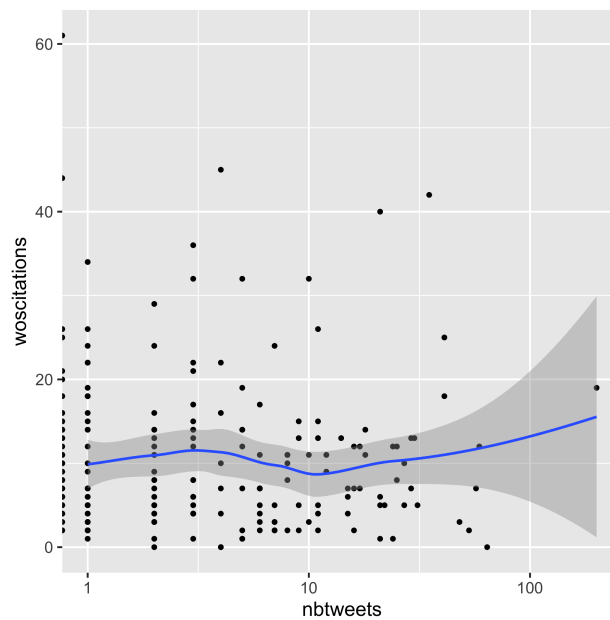
Scatterplots, add linear trend

```
citations_ecology %>%  
  ggplot() +  
  aes(x = nbtweets, y = woscitations) +  
  geom_point() +  
  geom_smooth(method = "lm") +  
  scale_x_log10()
```



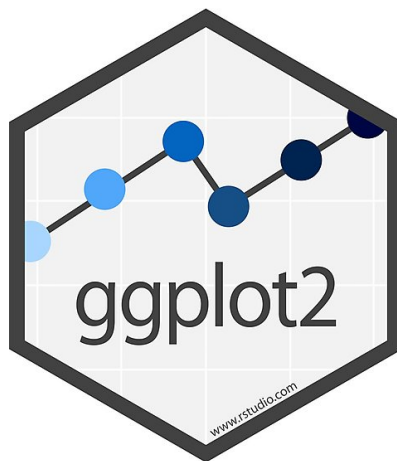
Scatterplots, add smoother

```
citations_ecology %>%  
  ggplot() +  
  aes(x = nbtweets, y = woscitations) +  
  geom_point() +  
  geom_smooth() +  
  scale_x_log10()
```



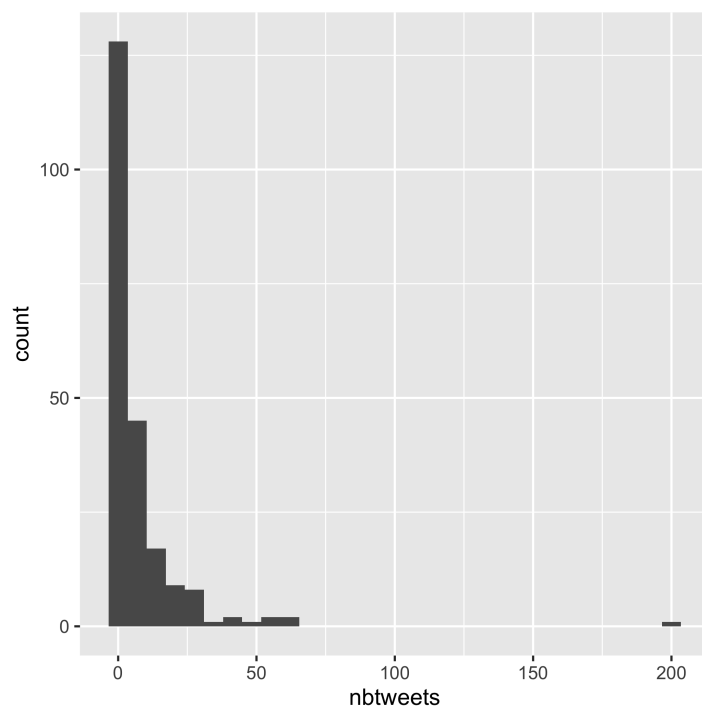
aes or not aes?

- If we are to establish a link between the values of a variable and a graphical feature, ie a mapping, then we need an aes().
- Otherwise, the graphical feature is modified irrespective of the data, then we do not need an aes().



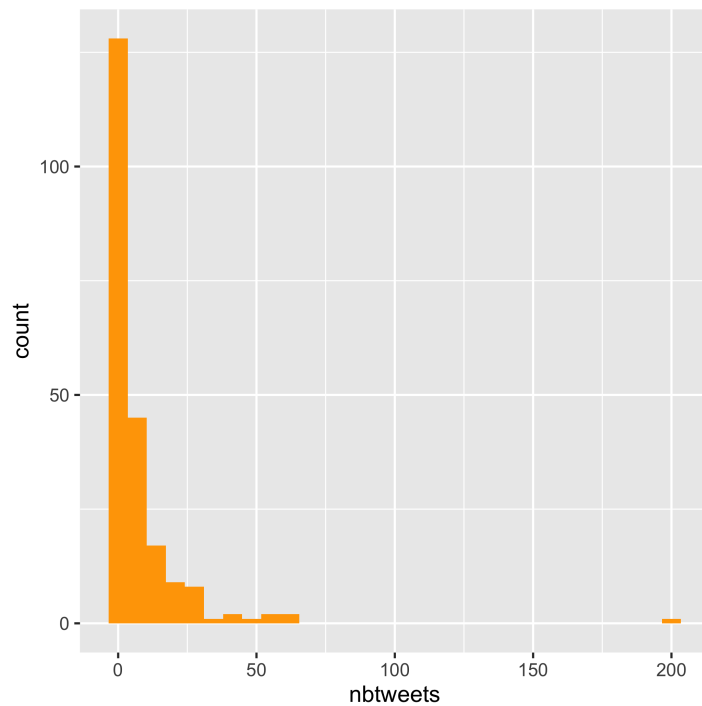
Histograms

```
citations_ecology %>%  
  ggplot() +  
  aes(x = nbtweets) +  
  geom_histogram()
```



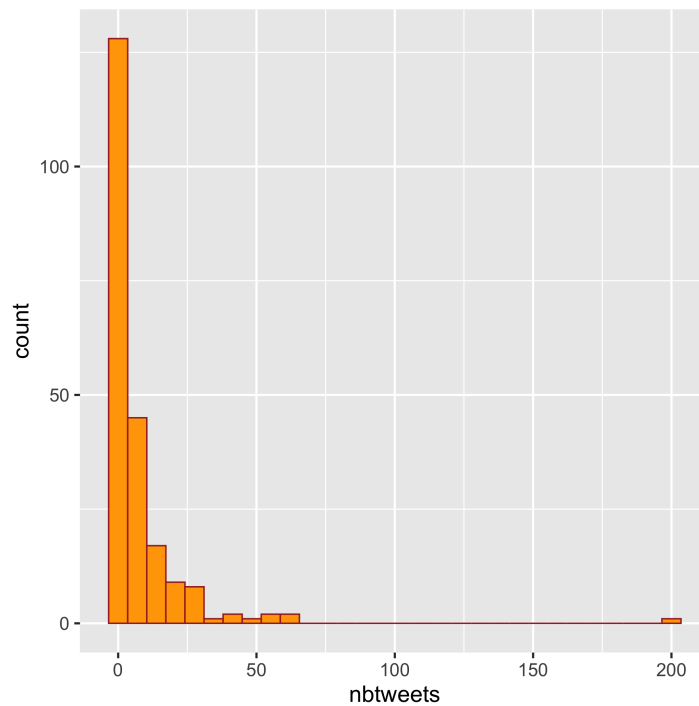
Histograms, with colors

```
citations_ecology %>%  
  ggplot() +  
  aes(x = nbtweets) +  
  geom_histogram(fill = "orange")
```



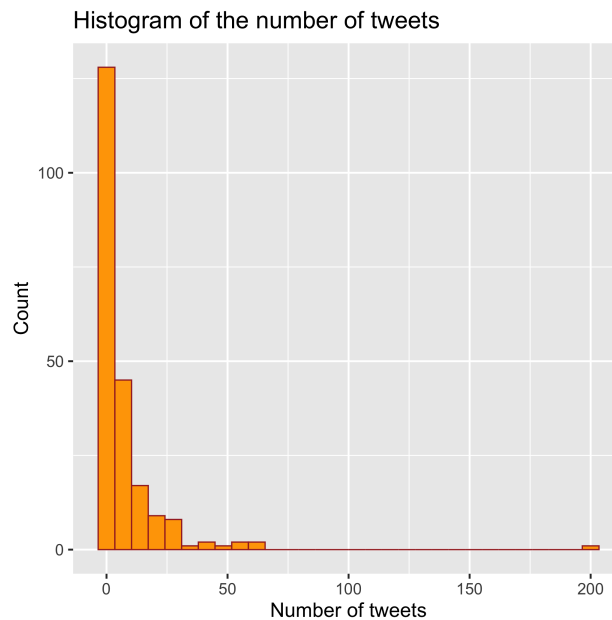
Histograms, with colors

```
citations_ecology %>%  
  ggplot() +  
  aes(x = nbtweets) +  
  geom_histogram(fill = "orange", color = "brown")
```



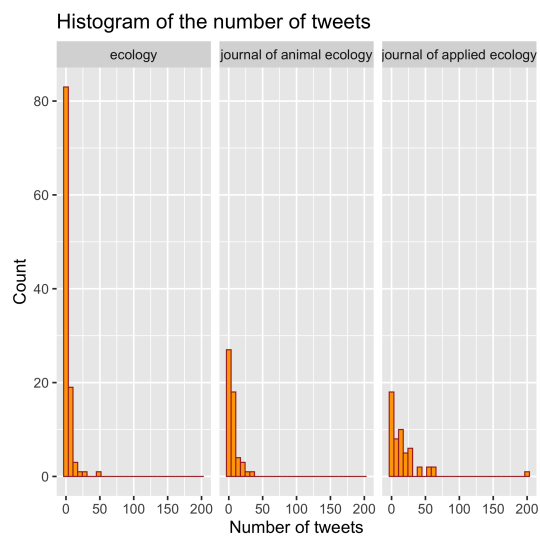
Histograms, with labels and title

```
citations_ecology %>%  
  ggplot() +  
  aes(x = nbtweets) +  
  geom_histogram(fill = "orange", color = "brown") +  
  labs(x = "Number of tweets",  
       y = "Count",  
       title = "Histogram of the number of tweets")
```



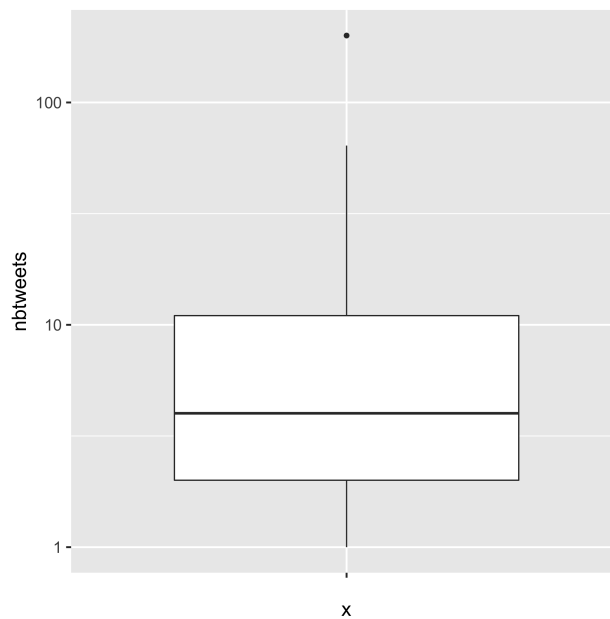
Histograms, by species

```
citations_ecology %>%  
  ggplot() +  
  aes(x = nbtweets) +  
  geom_histogram(fill = "orange", color = "brown") +  
  labs(x = "Number of tweets",  
       y = "Count",  
       title = "Histogram of the number of tweets") +  
  facet_wrap(vars(journal))
```



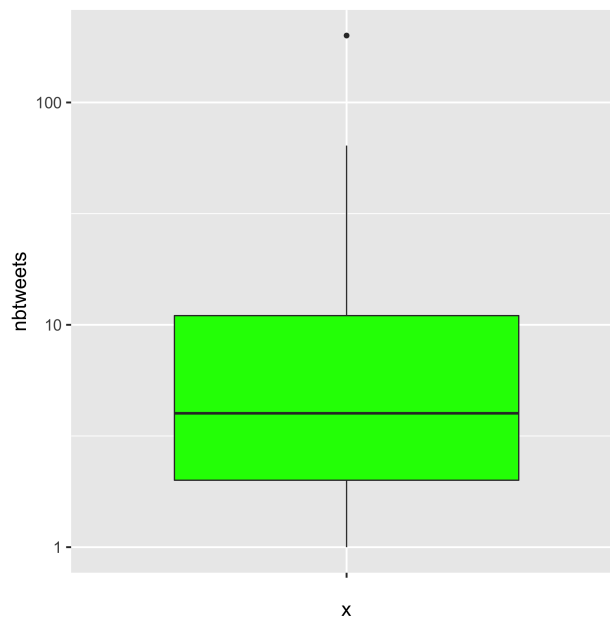
Boxplots

```
citations_ecology %>%  
  ggplot() +  
  aes(x = "", y = nbtweets) +  
  geom_boxplot() +  
  scale_y_log10()
```



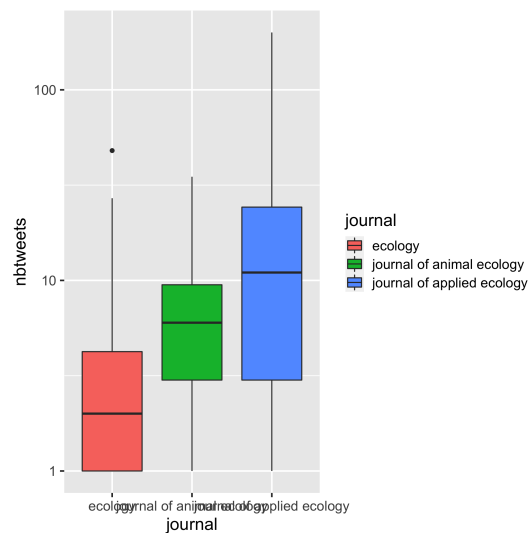
Boxplots with colors

```
citations_ecology %>%  
  ggplot() +  
  aes(x = "", y = nbtweets) +  
  geom_boxplot(fill = "green") +  
  scale_y_log10()
```



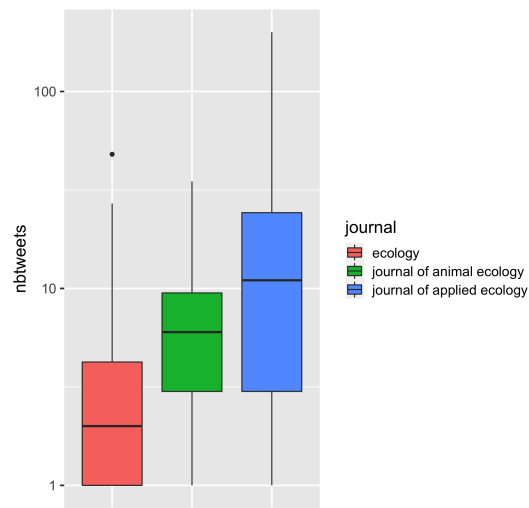
Boxplots with colors by species

```
citations_ecology %>%  
  ggplot() +  
  aes(x = journal, y = nbtweets, fill = journal) +  
  geom_boxplot() +  
  scale_y_log10()
```



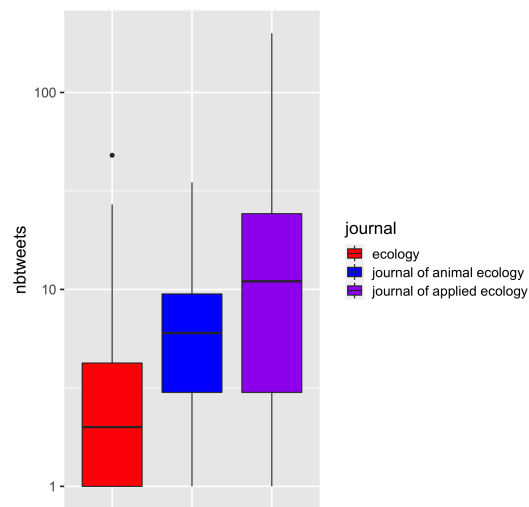
Get rid of the ticks on x axis

```
citations_ecology %>%  
  ggplot() +  
  aes(x = journal, y = nbtweets, fill = journal) +  
  geom_boxplot() +  
  scale_y_log10() +  
  theme(axis.text.x = element_blank()) +  
  labs(x = "")
```



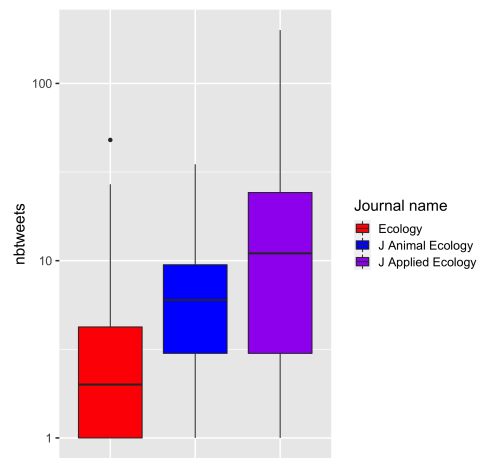
Boxplots, user-specified colors by species

```
citations_ecology %>%  
  ggplot() +  
  aes(x = journal, y = nbtweets, fill = journal) +  
  geom_boxplot() +  
  scale_y_log10() +  
  scale_fill_manual(  
    values = c("red", "blue", "purple")) +  
  theme(axis.text.x = element_blank()) +  
  labs(x = "")
```



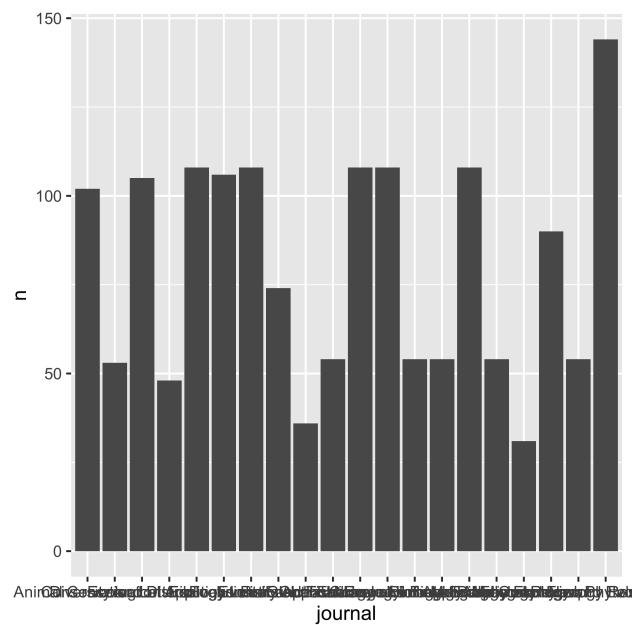
Boxplots, change legend settings

```
citations_ecology %>%  
  ggplot() +  
  aes(x = journal, y = nbtweets, fill = journal) +  
  geom_boxplot() +  
  scale_y_log10() +  
  scale_fill_manual(  
    values = c("red", "blue", "purple"),  
    name = "Journal name",  
    labels = c("Ecology", "J Animal Ecology", "J Applied Ecology")) +  
  theme(axis.text.x = element_blank()) +  
  labs(x = "")
```



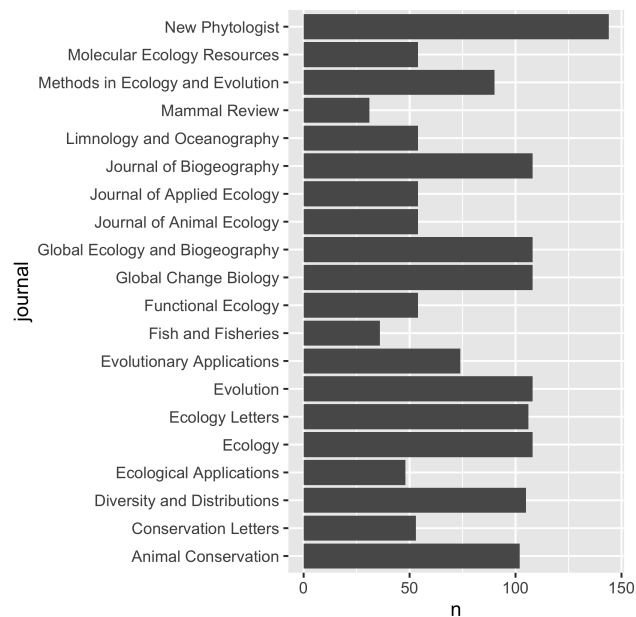
Ugly bar plots

```
citations %>%
  count(journal) %>%
  ggplot() +
  aes(x = journal, y = n) +
  geom_col()
```



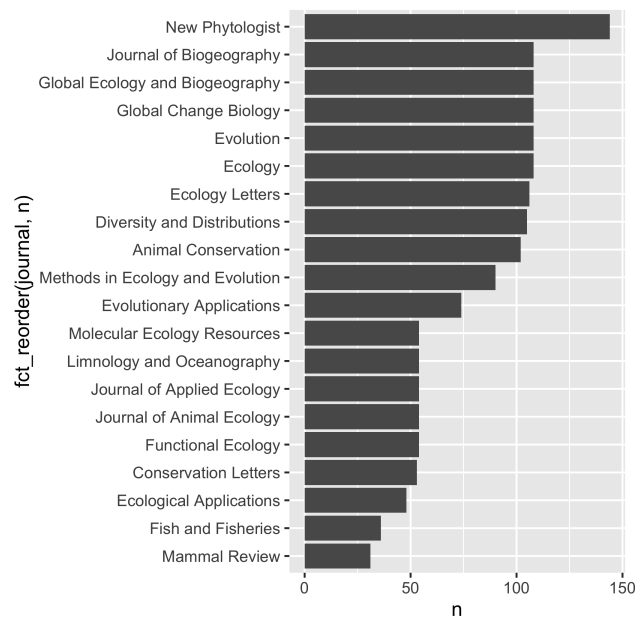
Idem, with flipping

```
citations %>%  
  count(journal) %>%  
  ggplot() +  
  aes(x = n, y = journal) +  
  geom_col()
```



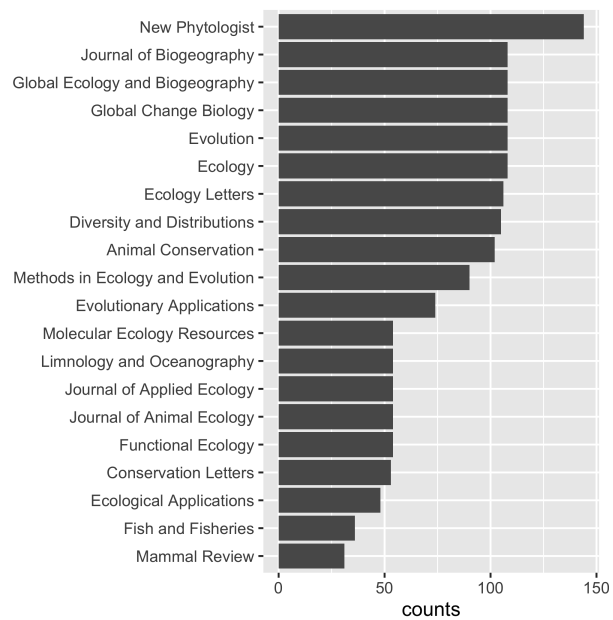
Idem, with factors reordering and flipping

```
citations %>%  
  count(journal) %>%  
  ggplot() +  
  aes(x = n, y = fct_reorder(journal, n)) +  
  geom_col()
```



Further cleaning

```
citations %>%  
  count(journal) %>%  
  ggplot() +  
  aes(x = n, y = fct_reorder(journal, n)) +  
  geom_col() +  
  labs(x = "counts", y = "")
```

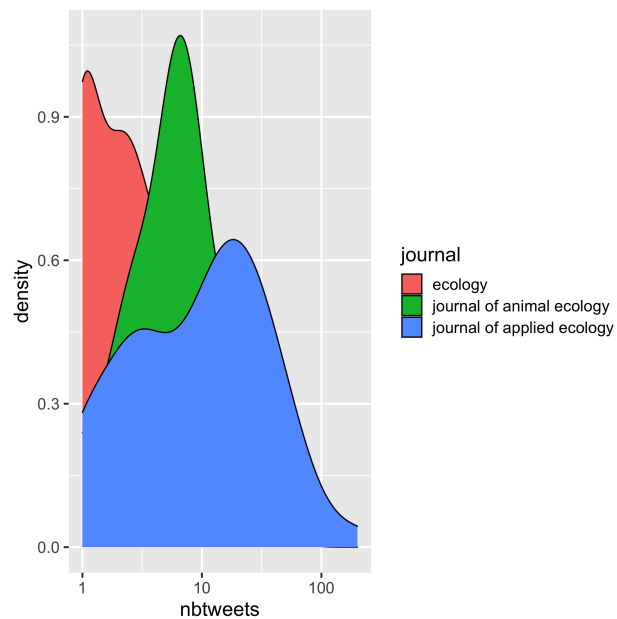


More about how to (tidy) work with factors

- Be the boss of your factors and
- forcats, forcats, vous avez dit forcats ?.

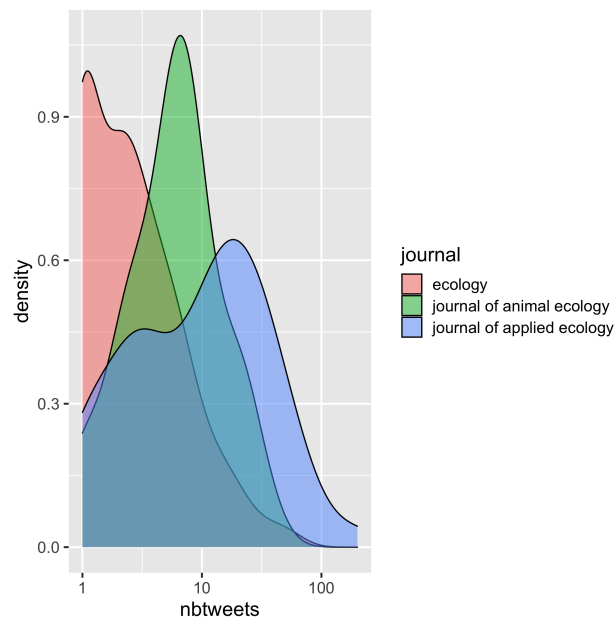
Density plots

```
citations_ecology %>%  
  ggplot() +  
  aes(x = nbtweets, fill = journal) +  
  geom_density() +  
  scale_x_log10()
```



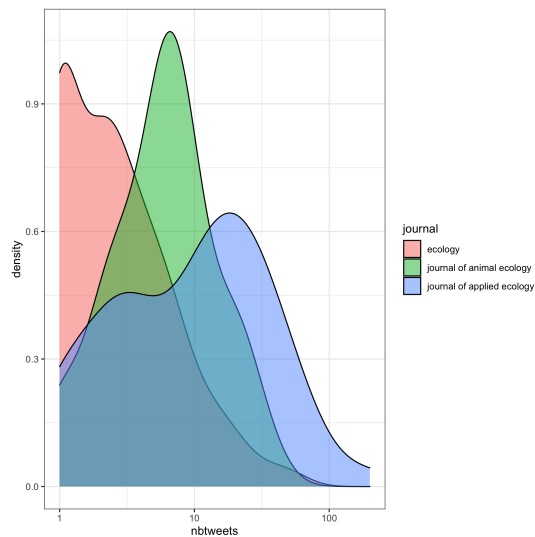
Density plots, control transparency

```
citations_ecology %>%  
  ggplot() +  
  aes(x = nbtweets, fill = journal) +  
  geom_density(alpha = 0.5) +  
  scale_x_log10()
```



Change default background B & W theme

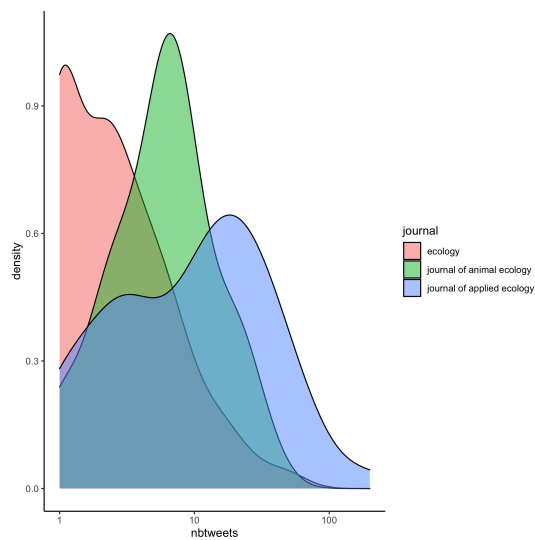
```
citations_ecology %>%  
  ggplot() +  
  aes(x = nbtweets, fill = journal) +  
  geom_density(alpha = 0.5) +  
  scale_x_log10() +  
  theme_bw()
```



Change default background theme

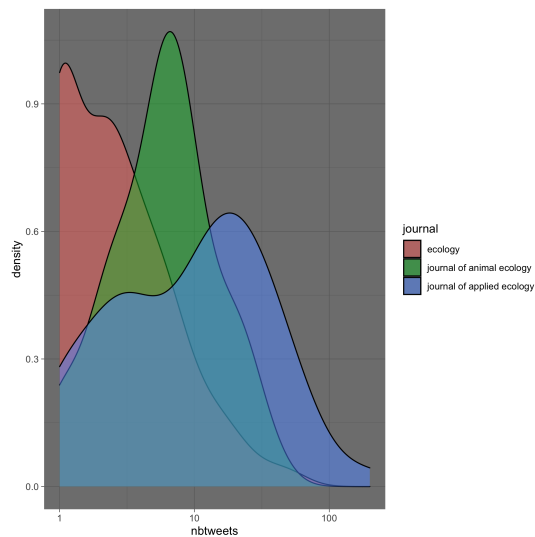
classic theme

```
citations_ecology %>%  
  ggplot() +  
  aes(x = nbtweets, fill = journal) +  
  geom_density(alpha = 0.5) +  
  scale_x_log10() +  
  theme_classic()
```



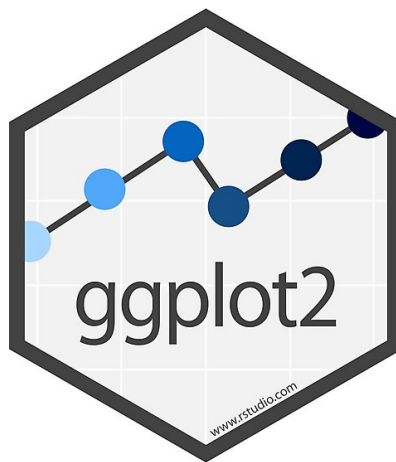
Change default background theme dark theme

```
citations_ecology %>%  
  ggplot() +  
  aes(x = nbtweets, fill = journal) +  
  geom_density(alpha = 0.5) +  
  scale_x_log10() +  
  theme_dark()
```



More on data visualisation with ggplot2

- [Portfolio](#) of ggplot2 plots
- [Cédric Scherer's portfolio](#) of data visualisations
- [Top](#) ggplot2 visualizations
- [Interactive](#) ggplot2 visualizations



To dive even deeper in the tidyverse

- **Learn the tidyverse**: books, workshops and online courses
- My selection of books:
 - **R for Data Science** et **Advanced R**
 - **Introduction à R et au tidyverse**
 - **Fundamentals of Data visualization**
 - **Data Visualization: A practical introduction**
- **Tidy Tuesdays videos** by D. Robinson chief data scientist at DataCamp
- Material of the **2-day workshop Data Science in the tidyverse** held at the RStudio 2019 conference
- Material of the stat545 course on **Data wrangling, exploration, and analysis with R** at the University of British Columbia
- List of best R packages (with their description) on **data import, wrangling and visualization**

How to switch from base R to tidyverse?

Couple of notes before we start. The list below is not exhaustive (best to read package documentation for that). For instance, it doesn't cover `lubridate` (which covers date/time related functions), `forcats` (which covers everything you would want to do to factors), `broom` (which tidies up messy R objects), `modelr` (which has helper functions for creating models) or `ggplot`. I also use data frame and tibble interchangeably, although they are obviously different.

Base R command	Tidyverse Command	What it does and why you should use the tidyverse version	Comment
<code>read.csv()</code>	<code>read_csv()</code>	reads in a csv file, but its much faster, shows progress bar for large files, can automatically parse data types	also see <code>read_delim()</code> , <code>read_tsv()</code> and <code>readxl::read_xlsx()</code>
<code>sort()</code> , <code>order()</code>	<code>arrange()</code>	sort column(n) within a data frame	see also <code>order_by()</code>
<code>mtcars\$mpg = ...</code>	<code>mutate()</code>	modify a column	see also <code>transmute()</code> which drops existing variables
<code>mtcars[,c("mpg", "am")]</code> , <code>subset()</code>	<code>select()</code> , <code>rename()</code>	select or rename columns	see also <code>pull()</code>
<code>mtcars[mtcars\$am == 1,]</code> , <code>subset()</code>	<code>filter()</code>	select rows based on a criterion	
<code>aggregate()</code>	<code>summarise()</code> , <code>summarize()</code> , <code>do()</code>	reduce grouped values to a single value	see also variants like <code>summarize_if()</code>
<code>ifelse()</code>	<code>if_else()</code> , <code>case_when()</code>	standard vectorized if else, but stricter than base version	see also <code>near()</code>
<code>unique()</code>	<code>distinct()</code>	finds unique rows in a data frame, but its much, faster	
<code>length(unique())</code>	<code>n_distinct()</code>	count the number of distinct values in a vector, faster	
<code>sample()</code> , <code>sample.int()</code>	<code>sample_n()</code> , <code>sample_frac()</code>	sample n rows or a fraction of rows from a dataframe	
<code>all.equal()</code>	<code>all_equal()</code>	checks if two vectors are the same	
<code>merge()</code>	<code>inner_join()</code> , <code>left_join()</code>	perform joins, much faster, verbose, and row order is maintain	see also <code>right_join()</code> , <code>full_join()</code> , <code>semi_join()</code> , <code>anti_join()</code>
<code>rbind()</code> , <code>cbind()</code>	<code>bind_rows()</code> , <code>bind_cols()</code>	concatenate two dataframes along rows or columns, much faster	
<code>x >= left & x <= right</code>	<code>between()</code>	easier to read and faster implementation for large datasets	see also <code>near()</code>
<code>nrow()</code> , <code>sum()</code>	<code>tally()</code> , <code>count()</code> , <code>add_tally()</code> , <code>add_count()</code>	count or sum up rows	
<code>c()</code>	<code>combine()</code>	combine into a vector	
extends base R	<code>cumall()</code> , <code>cumany()</code> , <code>cummean()</code>	extends base R collection of <code>cumsum()</code> , <code>cumprod()</code> etc	
<code>mtcars\$mpg[1,]</code> etc	<code>first()</code> , <code>last()</code> , <code>n()</code> , <code>top_n()</code>	works within groups, allows you to order by another column(s) and provide defaults for missing values	
<code>split()</code> , <code>aggregate()</code>	<code>group_by()</code>	create a grouped data frame (tibble) to perform operations on groups	see also <code>ungroup()</code>
<code>intersect()</code> , <code>union()</code>	<code>intersect()</code> , <code>union()</code>	set operations, but <code>dplyr</code> works on data frames as well	
<code>mtcars[mpg2 = c(NA, mtcars\$mpg[1:nrow(mtcars)-1])]</code>	<code>lead()</code> , <code>lag()</code>	No equivalent command in base R, easier to read	
<code>ifelse(..., NA)</code>	<code>na_if()</code>	convert a value to NA	
<code>switch()</code>	<code>recode()</code>	change certain values in your vector	see also <code>forcats</code> package when dealing with factors
<code>mtcars[3:5,]</code>	<code>slice()</code>	select rows based on row numbers	
<code>seq_along()</code> , <code>quantile()</code>	<code>row_number()</code> , <code>ntile()</code> , <code>min_rank()</code> etc	add rankings in various ways, much richer set of rankings supported than base R	
no easy way	<code>complete()</code> , <code>expand()</code>	expands the dataframe so that supplied columns are completely filled out	often used with <code>nesting()</code> , see also <code>full_seq()</code>
<code>expand.grid()</code>	<code>crossing()</code>	create a data frame of all possible combinations of supplied vectors	
<code>ifelse(is.na(...), ...)</code>	<code>drop_na()</code> , <code>replace_na()</code>	drop rows with missing values or convert NAs to supplied values	see also <code>fill()</code> , <code>coalesce()</code>
some mix of <code>paste/strsplit</code>	<code>separate()</code> , <code>unite()</code>	separate two columns based on regex or combine two columns into one	
<code>reshape2::dcast()</code>	<code>spread()</code>	convert long (tidy) data into wide (untidy) format	
<code>reshape2::melt()</code>	<code>gather()</code>	convert wide (untidy) data into long(tidy) format	
<code>replicate()</code>	<code>rerun()</code>	run an expression n number of times	
<code>unlist(lapply(x, f, n))</code>	<code>pluck()</code>	extract elements out of a list	
<code>lapply()</code> , <code>sapply()</code>	<code>map()</code> , <code>map2()</code>	apply a function to a set of values, working with lists	see also <code>map_chr()</code> , <code>map_lgl()</code> , <code>map_int()</code> , <code>map_dbl()</code> , <code>map_dff()</code>
<code>paste0()</code>	<code>glue()</code>	combine two strings together, but much more powerful because it allows for expressions	

The RStudio Cheat Sheets

Data Transformation with dplyr : : CHEAT SHEET



dplyr functions work with pipes and expect tidy data. In tidy data:

Each **variable** is in its own **column** & Each **observation**, or **case**, is in its own **row**  $x \%>\% f(y)$ becomes $f(x, y)$

Summarise Cases

These apply **summary functions** to columns to create a new table of summary statistics. Summary functions take vectors as input and return one value (see back).

summary function 

summarise(data, ...)
Compute table of summaries.
`summarise(mtcars, avg = mean(mpg))`

count(x, ..., wt = NULL, sort = FALSE)
Count number of rows in each group defined by the variables in ... Also **tally()**.
`count(iris, Species)`

VARIATIONS

summarise_all() - Apply funs to every column.
summarise_at() - Apply funs to specific columns.
summarise_if() - Apply funs to all cols of one type.

Group Cases

Use **group_by()** to create a "grouped" copy of a table. dplyr functions will manipulate each "group" separately and then combine the results.

 `mtcars %>%
group_by(cyl) %>%
summarise(avg = mean(mpg))`

group_by(data, ..., add = FALSE)
Returns copy of table grouped by ...
`g_iris <- group_by(iris, Species)`

ungroup(x, ...)
Returns ungrouped copy of table.
`ungroup(g_iris)`

Manipulate Cases

EXTRACT CASES

Row functions return a subset of rows as a new table.

 **filter(data, ...)** Extract rows that meet logical criteria. `filter(iris, Sepal.Length > 7)`

 **distinct(data, ..., keep_all = FALSE)** Remove rows with duplicate values.
`distinct(iris, Species)`

 **sample_frac(tbl, size = 1, replace = FALSE, weight = NULL, env = parent.frame())** Randomly select fraction of rows.
`sample_frac(iris, 0.5, replace = TRUE)`

 **sample_n(tbl, size, replace = FALSE, weight = NULL, env = parent.frame())** Randomly select size rows. `sample_n(iris, 10, replace = TRUE)`

 **slice(data, ...)** Select rows by position.
`slice(iris, 10:15)`

 **top_n(x, n, wt)** Select and order top n entries (by group if grouped data). `top_n(iris, 5, Sepal.Width)`

Logical and boolean operators to use with filter()

`<` `<=` `is.na()` `%in%` `|` `xor()`
`>` `>=` `!is.na()` `!` `&`

See ?base::logic and ?Comparison for help.

ARRANGE CASES

 **arrange(data, ...)** Order rows by values of a column or columns (low to high), use with **desc()** to order from high to low.
`arrange(mtcars, mpg)`
`arrange(mtcars, desc(mpg))`

ADD CASES

 **add_row(data, ..., before = NULL, after = NULL)**
Add one or more rows to a table.
`add_row(faithful, eruptions = 1, waiting = 1)`

Manipulate Variables

EXTRACT VARIABLES

Column functions return a set of columns as a new vector or table.

 **pull(data, var = 1)** Extract column values as a vector. Choose by name or index.
`pull(iris, Sepal.Length)`

 **select(data, ...)**
Extract columns as a table. Also **select_if()**.
`select(iris, Sepal.Length, Species)`

Use these helpers with **select()**,
e.g. `select(iris, starts_with("Sepal"))`

contains(match) **num_range(prefix, range)** ; e.g. `mpg:cyl`
ends_with(match) **one_of(...)** ; e.g. `-Species`
matches(match) **starts_with(match)**

MAKE NEW VARIABLES

These apply **vectorized functions** to columns. Vectorized funs take vectors as input and return vectors of the same length as output (see back).

vectorized function 

 **mutate(data, ...)**
Compute new column(s).
`mutate(mtcars, gpm = 1/mpg)`

 **transmute(data, ...)**
Compute new column(s), drop others.
`transmute(mtcars, gpm = 1/mpg)`

 **mutate_all(tbl, funs, ...)** Apply funs to every column. Use with **funs()**. Also **mutate_if()**.
`mutate_all(faithful, funs(log(), log2()))`
`mutate_if(iris, is.numeric, funs(log()))`

 **mutate_at(tbl, cols, funs, ...)** Apply funs to specific columns. Use with **funs()**, **vars()** and the helper functions for **select()**.
`mutate_at(iris, vars(-Species), funs(log()))`

 **add_column(data, ..., before = NULL, after = NULL)** Add new column(s). Also **add_count()**, **add_tally()**, **add_column(mtcars, new = 1:32)**

 **rename(data, ...)** Rename columns.
`rename(iris, Length = Sepal.Length)`



RStudio® is a trademark of RStudio, Inc. • CC BY SA RStudio • info@rstudio.com • 844-446-1212 • rstudio.com • Learn more with browseVignettes(package = "dplyr", "tidyverse") • dplyr 0.7.0 • tidyr 1.2.0 • Updated: 2017-03

8 Wednesday, October 11th

`runif(n, min, max)` creates random deviates of the uniform distribution

8.1 Programming Structures

In the past, we have seen `obj[]` and `func()` but now we will see `{}` for Programming Structures.

8.2 Compound Expressions

You can internal assignments in compound expressions. If you say

```
A = {  
  print("hello")  
  b=sqrt(100)  
  1:10  
}
```

outputs:

```
[1] 1 2 3 4 5 6 7 8 9 10.
```

Only assigned expressions can be referenced outside (and after) the compound expression.

8.3 If/Else

Unlike some other languages, R always requires braces.

The **else** also needs to be on the same line as the closing brace of the **if** – unless you surround the entire if-else with braces (so that you can treat it as one singular expression).

else if is another way to avoid having to have the braces.

Note that we will get to functions 2 weeks from now.

This will error:

```
if (y < 0) {  
  print("negative")  
}
```

is equiv to

```

if (y < 0) {
  print("negative")
} else {
  NULL
}

```

8.3.1 Vectorized ifelse

The above error can be solved using the vectorized `ifelse(cond, )`

```
ifelse(y < 0, yes="negative", no="non-negative")
```

8.3.2 Testing for missing values

Do NOT do:

```

z = NA
if (z == NA) {
  print("missing")
}

```

as it will error since `z == NA` is just `NA` which is not one of `TRUE` or `FALSE`. Instead, you *should* do:

```

z = NA
if (is.na(z)) {
  print("missing")
}

```

8.4 case_when

Something more practical:

R has a builtin `mtcars` dataset.

If we wanted to assign the mapping:

```

4 - cyl = "sm"
6 - cyl = "md"
8 - cyl = "lg"

```

```

dat |>
  mutate(size=case_when(

```

```
      cyl == 4 ~ "sm",  
      cyl == 6 ~ "md",  
      cyl == 8 ~ "lg"  
    ))
```

8.5 switch

```
switch(dat$cyl[1],  
      4 = "sm",  
      6 = "md",  
      8 = "lg"  
)
```

8.6 Lookup vector trick

```
lookup = c('4' = 'sm', '6' = 'md', '8' = 'lg')  
lookup[dat$cyl]
```

9 Wednesday, October 18th

9.1 Control Structures in R: Loops and Iterations

Control structures like loops are used to control the flow of execution in a script. In R, one of the most commonly used loops is the for loop.

9.2 For loops

The for loop allows us to repeat a set of commands for a specific number of times.

```
1 for (i in 1:5) {  
2   print(i)  
3 }
```

This code will print the numbers 1 through 5, once for each iteration of the loop.

9.2.1 Usage with vectors and lists

for loops can also iterate over vectors and lists.

```
1 fruits <- c("apple", "banana", "cherry")  
2 for (fruit in fruits) {  
3   print(fruit)  
4 }
```

Here, the loop iterates over each element of the fruits vector, printing the name of the fruit in each iteration.

9.2.2 Using the seq() function

The seq() function can be useful when you want to iterate over a sequence that doesn't increment by 1.

```
1 for (i in seq(1, 10, 2)) {  
2   print(i)  
3 }
```

This will print out the odd numbers between 1 and 10.

9.3 Control statements within loops

9.3.1 next

The next statement allows you to skip the rest of the current iteration and move on to the next one.

```
1 for (i in 1:5) {  
2   if (i == 3) {  
3     next  
4   }  
5   print(i)  
6 }
```

In this loop, the number 3 will not be printed because when i equals 3, the next statement will skip the rest of the loop body for that iteration.

9.3.2 break

The break statement allows you to exit the loop entirely.

```
1 for (i in 1:5) {  
2   if (i == 4) {  
3     break  
4   }  
5   print(i)  
6 }
```

In this example, the numbers 1, 2, and 3 will be printed. However, when i equals 4, the break statement will be executed, and the loop will terminate, so the numbers 4 and 5 will not be printed.

9.4 Apply Family Functions

The apply family of functions provides a suite of utilities to work on vectors, matrices, data frames, and lists without the need for explicit loops. Each function in this family is specialized to handle a specific data type and returns data in a specific format.

9.4.1 `apply()`

The `apply` function is primarily used for matrices or multi-dimensional arrays. It applies a function to the rows or columns of a matrix (or, more generally, to the margins of an array).

- **Syntax:** `apply(X, MARGIN, FUN, ...)`
 - `X`: the array or matrix
 - `MARGIN`: a vector indicating which margins should be “retained”. 1 indicates rows, 2 indicates columns.
 - `FUN`: the function to be applied
 - `...`: additional arguments to be passed to the function
- **Example:** To compute the mean of each column in a matrix:

```
1 apply(X = dat, MARGIN = 2, FUN = mean)
```

9.5 Anonymous Functions

In R, functions can be created on the fly without naming them—these are called anonymous functions. They are particularly useful when using functions like those in the `apply` family where you might only need the function temporarily.

9.5.1 Usage in `apply()`

Anonymous functions can be used directly within the `apply` function (or any of its relatives). The general structure is:

```
1 apply(data, margin, function(variable) expression_using_variable)
```

- **Example:** To compute the range (difference between max and min) of each column in a matrix:

```
1 apply(dat, 2, function(x) max(x) - min(x))
```

Here, the anonymous function calculates the difference between the maximum and minimum of `x`, where `x` represents each column in the matrix `dat`.

9.6 Range Computation Using `apply`

To compute the range (difference between maximum and minimum values) for each column of a dataset, we can define a custom function called `RANGE` and then utilize the `apply` function to compute it for each column.

```
1 RANGE = function(x, na.rm = FALSE) {  
2   max(x, na.rm = na.rm) - min(x, na.rm = na.rm)  
3 }
```

```
4
5 apply(dat2, 2, RANGE, na.rm = TRUE)
```

9.7 summarise

The `summarise` function in the `dplyr` package of the `tidyverse` is used to compute summary statistics of a given dataset. It's especially powerful when combined with the `group_by` function to obtain grouped summaries.

- **Basic Usage:** Summarise allows you to compute summary statistics such as means, medians, or custom aggregations.

```
1 summarise(dat, avg_height = mean(height), total_count = n())
```

- **With Grouping:** Using `group_by` before `summarise` lets you calculate summaries for each level of a categorical variable.

```
1 dat %>%
2   group_by(gender) %>%
3   summarise(avg_height = mean(height), total_count = n())
```

- **Multiple Summaries:** You can compute multiple summary statistics within a single call to `summarise`.

```
1 summarise(dat, avg_height = mean(height), median_height =
  ↪ median(height))
```

- **Using Custom Functions:** You can use custom functions within `summarise` to calculate desired metrics.

```
1 summarise(dat, range_height = max(height) - min(height))
```

- **NA Handling:** You can manage missing values by using the `na.rm` argument in your summary functions.

```
1 summarise(dat, avg_height = mean(height, na.rm = TRUE))
```

Note: The `summarise` function returns a new dataframe where each row represents a summarised value. The original dataframe remains unchanged.

9.8 while loops in R

The 'while' loop in R is used to execute a block of code repeatedly as long as a specified condition is 'TRUE'.

- **Basic Structure:** The structure of a 'while' loop starts with the 'while' keyword, followed by the condition in parentheses, and then the code to be executed within curly braces.

```
1 while (condition) {
2   # code to execute
3 }
```

- **Example:** A simple counter.

```

1     count <- 1
2     while (count <= 5) {
3         print(paste("Count is:", count))
4         count <- count + 1
5     }

```

- **Caution:** Ensure that the loop condition will eventually turn ‘FALSE’ to avoid infinite loops.
- **Breaking out of a loop:** The ‘break’ statement can be used to exit a ‘while’ loop prematurely.

```

1     count <- 1
2     while (TRUE) {
3         if (count > 5) {
4             break
5         }
6         print(paste("Count is:", count))
7         count <- count + 1
8     }

```

- **Skipping an iteration:** The ‘next’ statement allows you to skip the current iteration and proceed to the next one.

```

1     count <- 1
2     while (count <= 5) {
3         if (count == 3) {
4             count <- count + 1
5             next
6         }
7         print(paste("Count is:", count))
8         count <- count + 1
9     }

```

Note: While ‘while’ loops offer flexibility, they can be prone to errors like infinite loops. Always ensure your condition will eventually become ‘FALSE’, or have a mechanism (like ‘break’) to exit the loop.

10 Wednesday, October 25th

10.1 Variables in R

In R, variables can be broadly classified into global and local variables.

10.1.1 Global and Local Variables

Global variables exist in the global environment and can be accessed from anywhere in the script. On the other hand, local variables are only accessible within their respective functions or scopes.

```
1 tortilla = 8
2 carne = 7
3 make_taco = function(ingred1, ingred2) {
4   ingred1 + ingred2
5 }
6 make_taco(tortilla, carne)
```

In the code above, `tortilla` and `carne` are global variables, while `ingred1` and `ingred2` are local variables that act as arguments for the `make_taco()` function.

10.1.2 Lexical Scoping in R

R uses lexical scoping, which refers to determining the value of a variable based on how functions are defined, not when they are invoked. The term “lexical” denotes that scoping rules use a parse-time structure rather than a run-time structure.

Scoping is the process of determining the value of a variable. A variable’s scope is the section of the code where it exists and can be accessed.

R’s lexical scoping follows four main rules:

- Name masking
- Functions versus variables
- A fresh start
- Dynamic lookup

Local variables cannot be accessed from outside their function. Attempting to do so will result in an error message: “Error in eval(expr, envir, enclos): object ‘<object name>’ not found”.

Name Masking Names defined inside a function mask those defined outside. If a name isn’t found inside a function, R will look one level up. The same rules apply when a function is nested inside another function.

Functions vs. Variables If a function and a non-function have the same name (but exist in different environments), R can distinguish between them.

Fresh Start Local variables are reset every time a function is called.

Dynamic Lookup Dynamic lookup is R's mechanism for determining values when a function is executed, rather than when it's created.

10.2 Names and Values in R

10.2.1 Understanding Variable Bindings

Consider the following code:

```
1 a <- c(1, 2, 3)
```

Here, the variable **a** is bound to the numeric vector $\{1, 2, 3\}$.

```
1 typeof(a) # data type
2 lobstr::obj_size(a) # size
3 lobstr::obj_addr(a) # memory location
4 lobstr::ref(a) # tree of references
```

When two variables point to the same data, such as:

```
1 a <- c(1, 2, 3)
2 b <- a
```

The variable **b** does not create a copy of the data but references the same vector as **a**.

10.2.2 Copy on Modify

If the value of one of the objects is modified, R creates a copy of the original data for that object. This is evident when we modify **a** but not **b**.

```
1 a[1] <- 0
```

Now, **a** is $\{0, 2, 3\}$ and **b** remains $\{1, 2, 3\}$.

R uses a copy-on-modify (if necessary) strategy. Copying is only done when modifying the object. If modifications don't affect non-local variables, R can modify the object without copying.

10.2.3 Binding Counting in R

R uses a counting heuristic to track the number of names bound to a variable. It classifies bindings into:

- 0 - No bindings.
- 1 - One binding.
- 2+ - Multiple bindings (R doesn't distinguish the exact number beyond two).

When a name is unbound using the `rm()` function, R does not immediately free up the associated memory. Instead, R's garbage collection mechanism manages memory allocation.

10.3 Memory Management in R

R uses a Garbage Collection (GC) mechanism for memory management. When R detects an unbound name, it doesn't immediately free up the memory space. The `rm()` function only removes the binding, but the memory is managed by R's GC.

```
1 rm(b)
```

Thus, while the `rm()` function removes the binding between `b` and the numeric vector, it doesn't automatically free the memory space occupied by the numeric vector.

10.4 Advanced R Concepts

10.5 R's Motto: Everything is an Object

In R, everything that exists is considered an object, and all operations are a result of function calls. This includes arguments, values, and even the function and call themselves.

10.5.1 Objects and Functions in R

R has a unique way of treating operations. For example:

- Assignment:

```
1 w <- c(2, 4, 6)
2 # This can be seen as:
3 "<-"(w, c(2, 4, 6))
4 w
5 ## [1] 2 4 6
```

- Extraction:

```
1 w[2]
2 # Is equivalent to:
3 "["(w, 2)
4 ## [1] 4
```

- Replacement:

```
1 w[1] <- 0
2 # Can be represented as:
3 w = "<-"(w, 1, 0)
4 w
5 ## [1] 0 4 6
```

- Arithmetic Operations:

```
1 2 + 3
2 # Is actually a call to:
3 "+"(2, 3)
4 ## [1] 5
```

11 Wednesday, November 1st

11.1 Environments in R

Environments are key data structures in R that power scoping. They play a significant role in understanding lexical scoping, namespaces, and various evaluation aspects.

11.1.1 Understanding Environments

You can use the `rlang` package to explore and manipulate environments. For instance, the `env()` function from `rlang` can be used to create an environment:

```
1 e1 <- env(  
2   a = c(2, 4, 6),  
3   b = TRUE,  
4   c = "hi",  
5   d = 3.3  
6 )
```

11.1.2 Properties of Environments

Environments share some similarities with named lists, but with distinct features:

- Every name in an environment must be unique.
- The names in an environment are not ordered.
- Every environment has a parent.
- Environments are not duplicated when modified.

To view the content of an environment, rather than just its memory address, you can use the `env_print()` function:

```
1 env_print(e1)  
2 ## <environment: 0x7fb372f04550>  
3 ## parent: <environment: global>  
4 ## bindings:  
5 ## * a: <dbl>  
6 ## * b: <lgl>  
7 ## * c: <chr>  
8 ## * d: <dbl>
```

This behavior is similar to the distinction between `__str__` and `__repr__` in Python.

11.2 Characteristics of Environments

- Objects within environments are modified in-place, eliminating the need to create copies.
- Every environment has a parent, which is another environment. This parent-child relationship between environments is foundational to R's implementation of lexical scoping: if a name isn't found in an environment, R will search its parent, and so on.
- The only environment without a parent is the empty environment, denoted as `R_EmptyEnv`.
- Every environment's ancestry eventually leads to the empty environment.
- The Global Environment is a special environment. Its ancestors include every attached package.
- When a package is attached using `library()` or `require()`, it becomes a parent of the global environment. The immediate parent of the global environment is the most recently attached package.
- The sequence in which packages are attached is termed the “search path”. You can view these environments using `base::search()` or see the environments directly with `rlang::search_envs()`.

11.3 Environments and Function Execution

Every time a function is invoked in R, a new environment, termed the “execution environment,” is created to host its execution.

- The parent of the execution environment is the environment of the function, known as the “function environment.”
- Execution environments are typically ephemeral. Once the function finishes its execution, its environment is eligible for garbage collection.

11.3.1 Role and Importance of Environments

- During the execution of an expression in R, there's always a local, current, or active environment.
- R spawns a new environment each time a function is executed. Given that everything in R happens as a result of a function call, environments are frequently created and discarded.
- Functions in R often invoke other functions, leading to the creation and disposal of multiple environments in a nested manner.

11.3.2 Function Evaluation in R

Function calls in R are processed in three key steps:

1. The expressions of the arguments in the call (actual arguments) are matched to the formal arguments defined in the function.
2. A fresh environment is created. Within this environment, assignments are made for each formal argument, either containing the actual argument (if provided) or the default value (if the argument is missing and a default exists). The enclosing environment of this newly created environment is the environment where the function resides.

3. The body of the function is evaluated within this new environment. The result of this evaluation is returned as the function's output.

It's crucial to note a significant distinction in R's function evaluation process: the incorporation of the function's environment. This environment, which is where the function was originally defined, influences how names are resolved during execution. Consequently, R searches for names within this function environment.

12 Wednesday, November 8th

Everything that happens in R is an object. This is a foundational principle that influences how R handles computations and data structures. For example, when you perform an operation like $2 + 3$, it is actually interpreted as a function call.

$2 + 3$ is the same as `'+'(2, 3)`

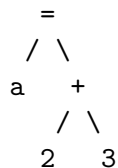
Tree Perspective of Expressions

Expressions in R can be represented as trees, where nodes represent operations and leaves represent operands or values. This tree structure helps in understanding how R parses and evaluates expressions.

Example 1: Simple Addition

Consider the expression `a = 2 + 3`. Its tree representation is as follows:

```
# Assigning the sum of 2 and 3 to a  
a <- 2 + 3
```



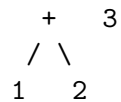
This tree shows the assignment operation (`'='`) with `'a'` as the left operand and the addition operation (`'+'`) as the right operand. The addition operation itself has `'2'` and `'3'` as its operands.

Example 2: Nested Addition

In a more complex expression like `1 + 2 + 3`, the tree structure helps us understand the order of operations:

```
# Adding 1, 2, and 3 sequentially  
1 + 2 + 3
```



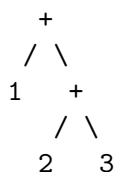


This tree indicates that R first adds 1 and 2, then adds the result to 3.

Example 3: Parenthesized Expressions

Parentheses can alter the default order of operations. For example, in $1 + (2 + 3)$, the addition inside the parentheses is performed first:

```
# Addition with parentheses
1 + (2 + 3)
```



This tree demonstrates that the expression ‘ $2 + 3$ ’ is evaluated first, followed by the addition of 1 to the result.

Understanding the object-oriented nature of R and how expressions are parsed into tree structures can greatly enhance your ability to write and debug R code. This perspective is especially useful for understanding complex expressions and ensuring accurate calculations.